

UNINTER
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

LINDOMAR JOSÉ BATISTÃO
RU 4427651

**DESENVOLVIMENTO DA ARQUITETURA DE UM SISTEMA MULTICANAL
PARA GESTÃO DE PEDIDOS DA REDE RAÍZES DO NORDESTE**

CAMPINAS
2026

Sumário

1. INTRODUÇÃO	6
2. ANÁLISE E REQUISITOS	7
2.1 Análise do Problema	7
2.2 Requisitos Funcionais	7
• Cliente	9
• Administrador	9
2.5 Benefícios Esperados	9
3 MODELAGEM E ARQUITETURA	10
3.1 Ênfase Escolhida	10
3.2 Arquitetura Geral do Sistema	10
3.3 Principais Endpoints da API	13
3.4 Tecnologias Utilizadas no Back-end	14
3.5 Persistência dos Dados	14
3.6 Segurança e Autenticação	14
3.7 Justificativa da Arquitetura	15
4 ENTREGA TÉCNICA	15
4.1 Tecnologias utilizadas	15
4.2 Estrutura do Projeto	16
4.3 Organização da API	17
4.4 Repositório do Projeto	18
4.5 Evidências da Implementação	18
4.5.1 – Back-end	18
5 PLANO DE TESTES E EVIDÊNCIAS DO BACKEND	19
5.1 Estratégia de Validação	19
5.2 Cadastro Inicial de Usuários	19
5.3 População Inicial do Banco de Dados	21
5.4 Listagem de usuários	23
5.5 Cadastro de Usuário	24
5.6 Autenticação do Usuário	25
5.7 Consulta do Usuário Logado	26
5.8 Listagem de Produtos	27
5.9 Listagem de Categorias	28
5.10 Listagem de Lojas	29
5.11 Criação de Pedido	30

5.12 Consulta dos Pedidos	31
5.13 Acesso sem Autenticação (Teste Negativo).....	32
5.14 Criação de Pedido sem Loja (Teste Negativo).....	33
5.15 Geração de token em ambiente hospedado na nuvem.....	34
5.17 Considerações Finais dos Testes no Back-end.....	36
6 PLANO DE TESTES E EVIDÊNCIAS DO FRONTEND WEB.....	37
7 PLANO DE TESTES E EVIDÊNCIAS DO FRONTEND MOBILE	44
8 PLANO DE TESTES E EVIDÊNCIAS.....	51
8.1 Estratégia de Validação.....	51
8.2 Cenários de Teste	51
8.3 Critérios de Aceitação	52
8.4 Evidências de Execução	53
8.6 Exemplo de Teste com Insomnia	53
Teste: Login de Usuário	53
Teste: Criação de Pedido.....	54
8.7 Considerações Sobre os Testes	54
9. CONCLUSÃO	55
10 REFERÊNCIAS BIBLIOGRÁFICAS.....	57
ANEXO A – Manual de Instalação.....	58
Back-end	58
Front-end Web	58
Front-end Mobile.....	58
Anexo B – Endpoints completos.....	59
Anexo C – Estrutura do Banco.....	60
Anexo D – Tecnologias.....	61
Anexo E – Mobile	62
Anexo F – WEB.....	63
Anexo F – Repositório	64

Índice de Ilustrações

Figura 1 - Arquitetura Geral do Sistema	10
Figura 2 - Diagrama de Caso de Uso de Cliente e Administrador	11
Figura 3 - Diagrama de Classes do Projeto	12
Figura 4 - Diagrama Entidade-Relacionamento	13
Figura 5 - Estrutura de diretórios do back-end do sistema.	17
Figura 6 - Back-end em funcionamento	18
Figura 7 - Back-end em funcionamento no browser	19
Figura 8 - Criando usuários.	20
Figura 9 - Tabela de usuários já populada	21
Figura 10 - Populando todas as tabelas do banco	23
Figura 11 - Listagem de usuários utilizando o Insomnia.	24
Figura 12 - Cadastro de usuário utilizando o endpoint de criação da API.	25
Figura 13 - Geração do token JWT utilizando o Insomnia.	26
Figura 14 - Consulta dos dados do usuário autenticado por meio do endpoint /api/usuarios/me/.	27
Figura 15 - Listagem de produtos por meio do endpoint /api/produtos/.	28
Figura 16 - Listagem das categorias cadastradas por meio do endpoint /api/categorias/.	29
Figura 17 - – Listagem das lojas cadastradas por meio do endpoint /api/lojas/.	30
Figura 18 - Criação de pedido por meio do endpoint /api/pedidos/.	31
Figura 19 - Consulta dos pedidos por meio do endpoint /api/pedidos/.	32
Figura 20 - Tentativa de acesso ao endpoint /api/pedidos/ sem autenticação.	33
Figura 21 - Tentativa de criação de pedido sem informar o campo obrigatório "loja".	34
Figura 22 - Geração de token JWT utilizando a API hospedada em ambiente de nuvem.	35
Figura 23 - Front-end em funcionamento	37
Figura 24 - Tela de login no front	37
Figura 25 - Front-end em funcionamento no browser com usuário comum logado	38
Figura 26 - Página de produtos	38
Figura 27 - Página de pedidos	39
Figura 28 - Página de históricos de pedidos	39
Figura 29 - Página do clube de fidelidade	40
Figura 30 - Dashboard de administrador do sistema	40
Figura 31 - Página de visualização dos usuários pelo administrador	41
Figura 32 - Página de visualização de categorias pelo administrador	41
Figura 33 - Página de visualização de estoque pelo administrador	42
Figura 34 - Página de visualização de pedidos pelo administrador	42
Figura 35 - Página de visualização de pagamentos pelo administrador	43
Figura 36 - Página de visualização de fidelidades pelo administrador	43
Figura 37 - Iniciando o mobile utilizando Expo	44
Figura 38 - Tela de login	44
Figura 39 - Tela do carrinho do usuário	45
Figura 40 - Tela de produtos do usuário	45
Figura 41 - Tela do perfil do usuário	46
Figura 42 - Login como administrador	46
Figura 43 - Tela principal administrativa	47
Figura 44 - Tela de usuários	47

Figura 45 - Tela de categorias	48
Figura 46 - Tela de produtos	48
Figura 47 - Tela de estoque	49
Figura 48 - Tela de pedidos	49
Figura 49 - Tela de pagamentos	50
Figura 50 - Tela de fidelidades.....	50

Índice de Tabelas

Tabela 1 - Lista de Requisitos Funcionais.	8
Tabela 2 - Lista de Requisitos Não Funcionais.....	9
Tabela 3 - Métodos e Endpoints do projeto.	14
Tabela 4 - Tecnologias utilizadas.....	14
Tabela 5 - Tecnologias empregadas no desenvolvimento do sistema.	15
Tabela 6 - Módulos do Projeto	18
Tabela 7 - Cenários de Teste	52

1. INTRODUÇÃO

O crescimento de uma empresa normalmente traz novos desafios relacionados à organização de seus processos e ao gerenciamento das informações. À medida que aumenta o número de clientes, unidades de atendimento e canais de venda, torna-se necessário utilizar soluções tecnológicas capazes de integrar essas operações de forma eficiente, garantindo maior agilidade, segurança e facilidade na tomada de decisões.

A rede de restaurantes Raízes do Nordeste representa esse cenário. O negócio, que teve início como uma lanchonete familiar, expandiu suas atividades e passou a atender clientes por diferentes canais, como lojas físicas, aplicação web, aplicativo mobile, totens de autoatendimento e serviços de delivery. Entretanto, essa expansão também aumentou a complexidade do gerenciamento dos pedidos, do controle de produtos, do estoque e das informações compartilhadas entre os diferentes setores da empresa.

Diante desse contexto, este trabalho propõe o desenvolvimento da arquitetura de um sistema multicanal para apoiar a gestão integrada da rede. A solução foi projetada utilizando uma API REST desenvolvida com Django, responsável por centralizar as regras de negócio e disponibilizar os dados para diferentes interfaces, incluindo uma aplicação web desenvolvida em React e um aplicativo mobile desenvolvido em React Native. Dessa forma, todos os canais passam a utilizar a mesma base de informações, reduzindo inconsistências e facilitando a manutenção do sistema.

Além do desenvolvimento da solução, o projeto contempla a análise dos requisitos, a modelagem do banco de dados, a definição da arquitetura da aplicação, a implementação dos principais módulos do sistema e a realização de testes funcionais para validação das funcionalidades desenvolvidas. Todo o processo foi documentado com o objetivo de demonstrar não apenas o funcionamento da aplicação, mas também as decisões técnicas adotadas durante o desenvolvimento.

Por fim, este trabalho busca aplicar, de forma integrada, os conhecimentos adquiridos ao longo do curso de Análise e Desenvolvimento de Sistemas, aproximando a proposta das situações encontradas no mercado de trabalho. A solução apresentada procura atender às necessidades identificadas no estudo de caso, fornecendo uma arquitetura organizada, escalável e preparada para futuras evoluções do sistema.

2. ANÁLISE E REQUISITOS

2.1 Análise do Problema

A rede de restaurantes Raízes do Nordeste encontra-se em processo de expansão e necessita de uma solução tecnológica capaz de suportar múltiplos canais de atendimento, incluindo lojas físicas, aplicativo mobile, portal web e totens de autoatendimento. O sistema atualmente utilizado não atende mais às necessidades operacionais da empresa, dificultando o gerenciamento dos pedidos, o controle das informações e a integração entre os diversos canais de venda.

Além disso, a organização necessita de mecanismos que permitam o gerenciamento de usuários, produtos, pedidos, fidelização de clientes e integração com meios de pagamento, mantendo a disponibilidade do sistema mesmo em períodos de alta demanda, como horários de almoço e jantar.

Dessa forma, a solução proposta busca fornecer uma plataforma centralizada para gestão dos processos de venda, permitindo escalabilidade, segurança e facilidade de manutenção.

2.2 Requisitos Funcionais

Os requisitos funcionais representam as funcionalidades que o sistema deve oferecer aos usuários.

ID	Requisito Funcional	Descrição
RF01	Cadastro de Usuários	O sistema deve permitir o cadastro de novos clientes.
RF02	Autenticação	O sistema deve permitir login e logout dos usuários.
RF03	Gerenciamento de Perfil	O usuário deve poder consultar seus dados cadastrais.
RF04	Listagem de Produtos	O sistema deve exibir os produtos disponíveis para venda.
RF05	Consulta de Categorias	O sistema deve permitir visualizar produtos organizados por categoria.
RF06	Carrinho de Compras	O usuário deve poder adicionar produtos ao carrinho.
RF07	Alteração de Quantidades	O usuário deve poder aumentar ou diminuir a quantidade dos produtos no carrinho.
RF08	Remoção de Produtos	O usuário deve poder remover itens do carrinho.

RF09	Finalização de Pedido	O sistema deve permitir concluir a compra dos produtos selecionados.
RF10	Consulta de Pedidos	O cliente deve visualizar seus pedidos realizados.
RF11	Gerenciamento de Pedidos	O administrador deve acompanhar os pedidos realizados.
RF12	Controle de Status	O sistema deve controlar os status dos pedidos (Criado, Pago, Em Preparo, Pronto e Entregue).
RF13	Cadastro de Lojas	O sistema deve permitir o gerenciamento das unidades da rede.
RF14	Programa de Fidelidade	O sistema deve possibilitar o acúmulo de benefícios para clientes cadastrados.
RF15	Integração Multicanal	O sistema deve registrar pedidos originados de aplicativo, site, balcão e totens.

Tabela 1 - Lista de Requisitos Funcionais.

2.3 Requisitos Não Funcionais

Os requisitos não funcionais definem características de qualidade e restrições da solução.

ID	Requisito Não Funcional	Descrição
RNF01	Desempenho	O sistema deve suportar múltiplos acessos simultâneos sem degradação significativa.
RNF02	Disponibilidade	O sistema deve permanecer disponível durante horários de pico de atendimento.
RNF03	Segurança	O acesso às funcionalidades restritas deve ocorrer mediante autenticação de usuário.
RNF04	Proteção de Dados	O sistema deve respeitar os princípios da LGPD para tratamento de dados pessoais.
RNF05	Escalabilidade	A solução deve permitir crescimento futuro da rede sem necessidade de reestruturação completa.
RNF06	Responsividade	As interfaces web devem se adaptar a diferentes tamanhos de tela.

RNF07	Portabilidade	O aplicativo mobile deve funcionar em dispositivos Android.
RNF08	Manutenibilidade	O código deve seguir boas práticas de desenvolvimento para facilitar futuras manutenções.
RNF09	Integração	A comunicação entre os módulos deve ocorrer por meio de APIs REST.
RNF10	Persistência dos Dados	As informações devem ser armazenadas em banco de dados relacional.
RNF11	Confiabilidade	O sistema deve garantir a integridade das informações armazenadas.
RNF12	Usabilidade	As interfaces devem apresentar navegação simples e intuitiva para os usuários.

Tabela 2 - Lista de Requisitos Não Funcionais.

2.4 Usuários do Sistema

O sistema possui dois perfis principais de usuários:

- Cliente

Responsável por realizar cadastro, efetuar login, consultar produtos, montar carrinho de compras, realizar pedidos e acompanhar o histórico de compras.

- Administrador

Responsável pelo gerenciamento dos produtos, lojas, pedidos e acompanhamento das operações realizadas no sistema.

2.5 Benefícios Esperados

Com a implantação da solução proposta, espera-se:

- Centralizar o gerenciamento dos pedidos da rede;
- Melhorar a experiência dos clientes;
- Facilitar o controle operacional das lojas;
- Possibilitar futuras integrações com meios de pagamento;
- Permitir expansão sustentável do negócio;
- Disponibilizar uma base tecnológica preparada para crescimento da empresa.

3 MODELAGEM E ARQUITETURA

3.1 Ênfase Escolhida

A ênfase escolhida para este projeto é **Back-end**, pois o foco principal da solução está na construção da API, modelagem do banco de dados, autenticação dos usuários, regras de negócio e integração entre os canais de atendimento da rede Raízes do Nordeste.

A arquitetura proposta permite que diferentes interfaces, como aplicação web, aplicativo mobile, balcão e totens de autoatendimento, utilizem a mesma base de dados e os mesmos serviços por meio de uma API REST.

3.2 Arquitetura Geral do Sistema

O sistema utiliza uma arquitetura cliente-servidor, na qual as aplicações front-end consomem os dados disponibilizados pela API back-end.

Fluxo geral da solução:



Figura 1 - Arquitetura Geral do Sistema

Fonte: Elaborado pelo autor com ícones da plataforma Icons8 (2026).

A aplicação mobile foi desenvolvida em React Native com Expo, enquanto a API foi implementada utilizando Django REST Framework. A comunicação entre as interfaces e o servidor ocorre por meio de requisições HTTP, com troca de dados no formato JSON.

3.3 Diagrama de Caso de Uso

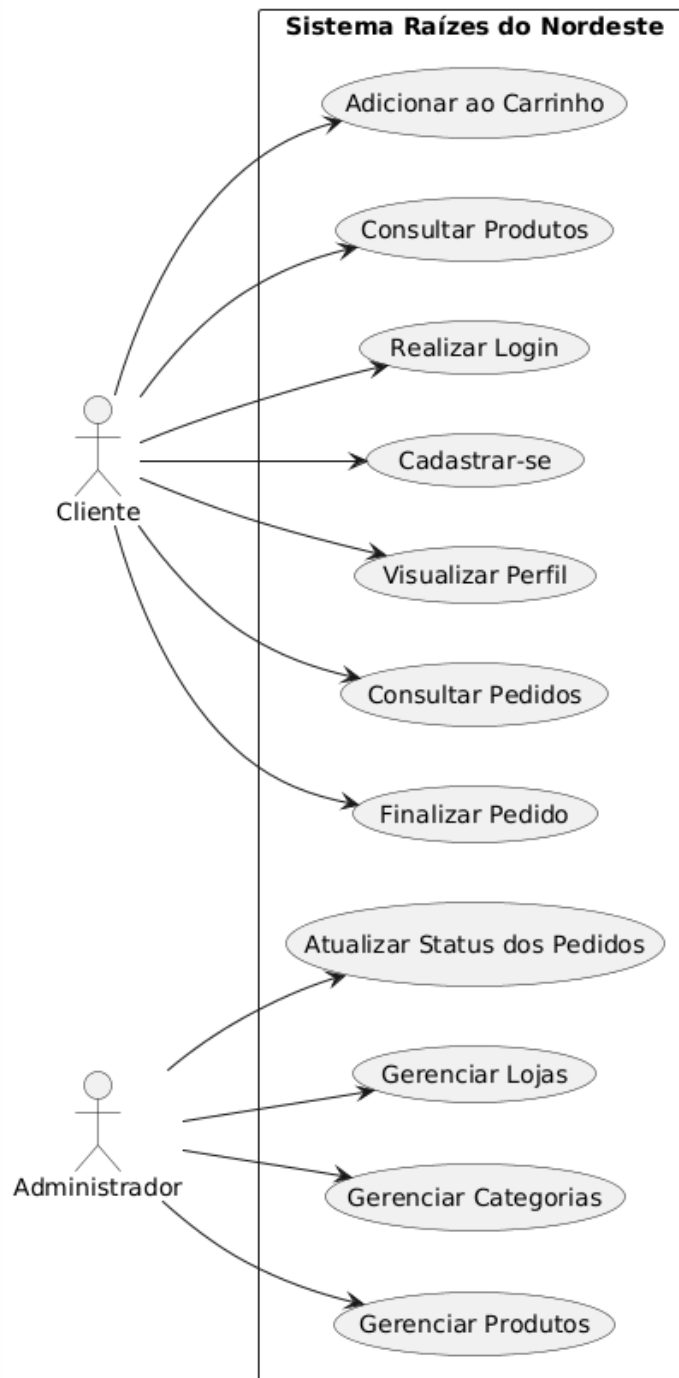


Figura 2 - Diagrama de Caso de Uso de Cliente e Administrador

Fonte: Elaborado pelo autor (2026).

3.4 Diagrama de Classes

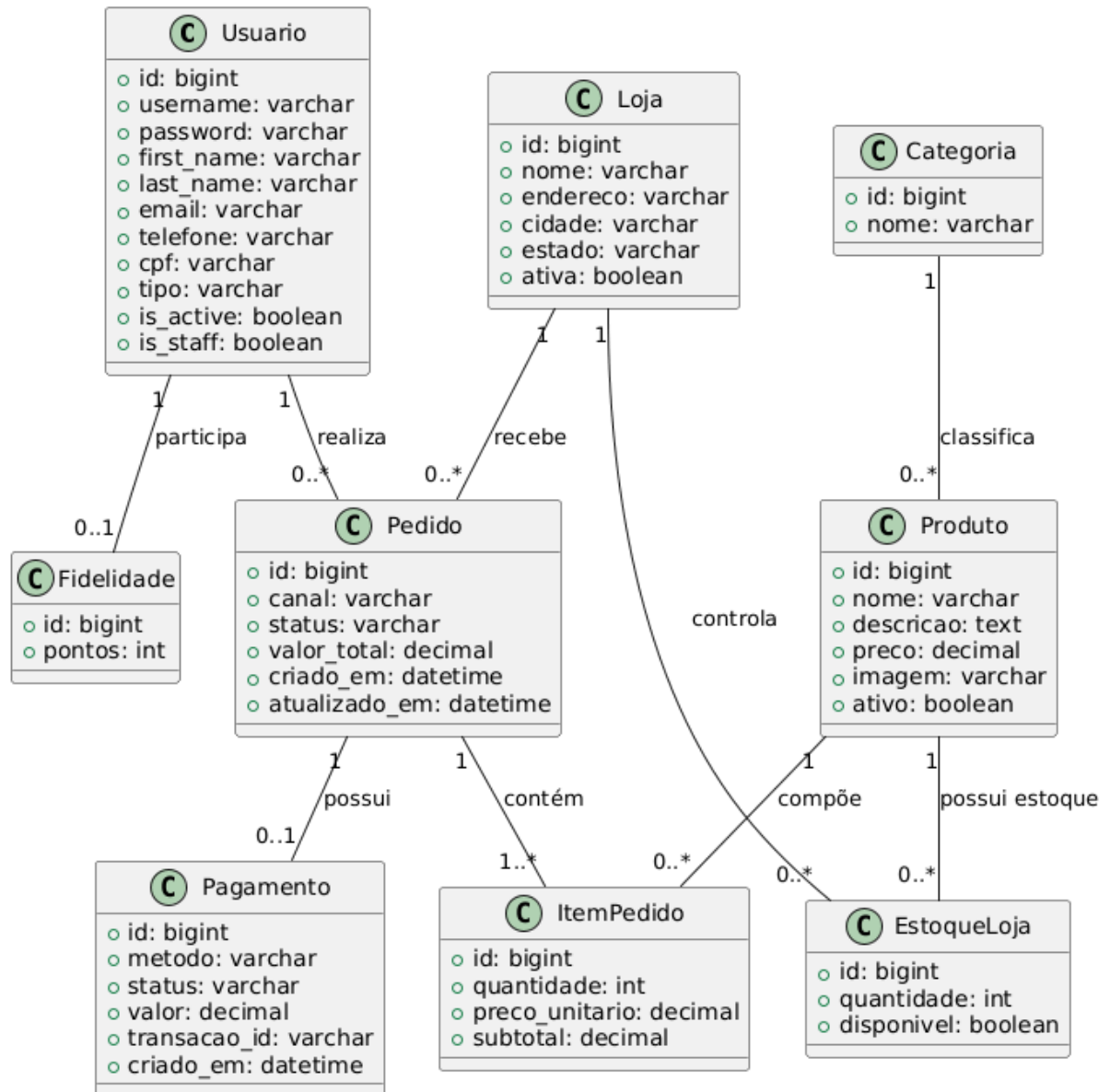


Figura 3 - Diagrama de Classes do Projeto

Fonte: Elaborado pelo autor (2026).

3.5 DER - Diagrama Entidade-Relacionamento

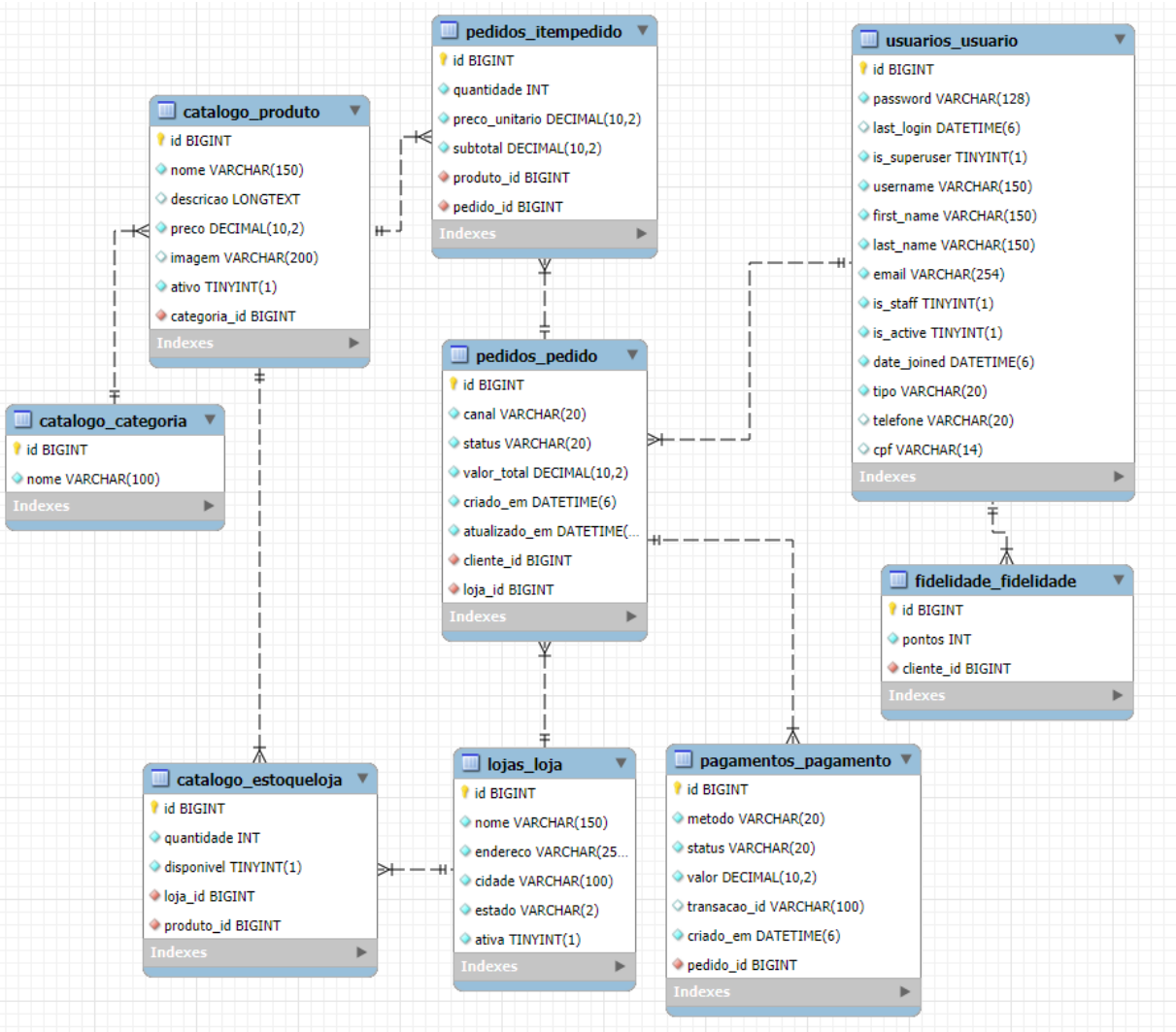


Figura 4 - Diagrama Entidade-Relacionamento

Fonte: Elaborado pelo autor (2026).

3.3 Principais Endpoints da API

A API foi desenvolvida seguindo o padrão REST, permitindo a comunicação entre o back-end e as aplicações cliente.

Método	Endpoint	Descrição
POST	/api/token/	Realiza autenticação do usuário e retorna o token JWT
POST	/api/token/refresh/	Atualiza o token de autenticação
POST	/api/usuarios/	Cadastra um novo usuário
GET	/api/usuarios/me/	Retorna os dados do usuário logado
GET	/api/produtos/	Lista os produtos cadastrados

GET	/api/categorias/	Lista as categorias dos produtos
GET	/api/lojas/	Lista as lojas cadastradas
GET	/api/pedidos/	Lista os pedidos do usuário logado
POST	/api/pedidos/	Cria um novo pedido
GET	/api/pedidos/{id}/	Consulta os detalhes de um pedido
PUT	/api/pedidos/{id}/	Atualiza informações de um pedido
DELETE	/api/pedidos/{id}/	Remove um pedido, quando permitido

Tabela 3 - Métodos e Endpoints do projeto.

3.4 Tecnologias Utilizadas no Back-end

Tecnologia	Finalidade
Python	Linguagem principal do back-end
Django	Framework web utilizado no desenvolvimento do sistema
Django REST Framework	Criação da API REST
Simple JWT	Autenticação baseada em tokens JWT
SQLite	Banco de dados utilizado no ambiente de desenvolvimento
MySQL	Banco de dados previsto para ambiente de produção
Django ORM	Mapeamento objeto-relacional e persistência dos dados

Tabela 4 - Tecnologias utilizadas.

3.5 Persistência dos Dados

A persistência dos dados foi implementada utilizando banco de dados relacional. Durante o desenvolvimento, foi utilizado o SQLite por sua simplicidade de configuração e facilidade de testes locais. Entretanto, a estrutura do sistema permite migração para bancos mais robustos, como MySQL, em ambiente de produção.

O Django ORM foi utilizado para representar as tabelas do banco por meio de classes Python, facilitando operações de criação, consulta, atualização e exclusão de registros.

3.6 Segurança e Autenticação

A autenticação do sistema foi implementada utilizando JWT. Após realizar login, o usuário recebe um token de acesso, que deve ser enviado nas requisições protegidas da API.

Exemplo de cabeçalho de autenticação:

`Authorization: Bearer token_de_acesso`

Essa abordagem garante que funcionalidades como finalização de pedido, consulta de perfil e listagem de pedidos sejam acessadas apenas por usuários autenticados.

3.7 Justificativa da Arquitetura

A escolha por uma API REST permite separar o back-end das interfaces de usuário, tornando possível que diferentes canais consumam os mesmos dados e regras de negócio. Dessa forma, a aplicação web, o aplicativo mobile e futuros totens podem utilizar a mesma estrutura de serviços.

Essa arquitetura favorece a escalabilidade, manutenção e evolução do sistema, pois novas funcionalidades podem ser adicionadas ao back-end sem a necessidade de reconstruir todas as interfaces do usuário.

4 ENTREGA TÉCNICA

4.1 Tecnologias utilizadas

Tecnologia	Finalidade
Python 3	Linguagem de programação
Django	Desenvolvimento da API
Django REST Framework	Construção dos endpoints REST
SQLite	Banco de dados
React	Aplicação Web
React Native	Aplicação Mobile
Git	Controle de versão
GitHub	Hospedagem do código
Insomnia	Testes da API

Tabela 5 - Tecnologias empregadas no desenvolvimento do sistema.

4.2 Estrutura do Projeto

O back-end do sistema foi desenvolvido utilizando o framework **Django**, seguindo uma organização modular para facilitar a manutenção e a separação das responsabilidades da aplicação. A pasta principal **api** contém os módulos responsáveis pelas funcionalidades do sistema, enquanto a pasta **raizes** reúne os arquivos de configuração do projeto.

Os principais diretórios são:

- **apps/**: contém os módulos funcionais da aplicação, organizados em:
 - **catalogo**: gerenciamento de produtos, categorias e estoque;
 - **usuarios**: cadastro, autenticação e gerenciamento de usuários;
 - **pedidos**: controle de pedidos e itens dos pedidos;
 - **pagamentos**: gerenciamento dos pagamentos realizados;
 - **lojas**: cadastro e gerenciamento das unidades da rede;
 - **fidelidade**: controle do programa de fidelidade dos clientes.
- **core/**: contém arquivos compartilhados entre os módulos da aplicação.
- **migrations/**: armazena as migrações responsáveis pela criação e atualização da estrutura do banco de dados.
- **raizes/**: contém as configurações gerais do projeto, incluindo arquivos de configuração, rotas principais e inicialização do servidor.
- **manage.py**: utilitário utilizado para executar comandos administrativos do Django, como criação de migrações, inicialização do servidor e gerenciamento da aplicação.
- **requirements.txt**: arquivo que lista todas as dependências necessárias para execução do projeto.

Essa organização modular permite que cada funcionalidade seja desenvolvida de forma independente, tornando o sistema mais organizado, reutilizável e de fácil manutenção.

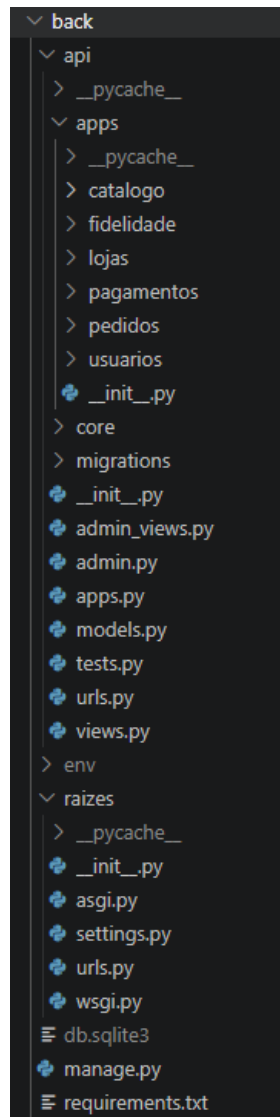


Figura 5 - Estrutura de diretórios do back-end do sistema.

Fonte: Elaborado pelo autor (2026).

4.3 Organização da API

A API foi dividida em módulos independentes, seguindo a arquitetura proposta pelo Django. Cada módulo possui responsabilidades específicas, facilitando a manutenção do sistema.

Módulo	Responsabilidade
Usuários	Cadastro e autenticação
Catálogo	Produtos e categorias
Pedidos	Cadastro dos pedidos
Pagamentos	Controle dos pagamentos
Fidelidade	Programa de pontos
Lojas	Cadastro das unidades

Tabela 6 - Módulos do Projeto

4.4 Repositório do Projeto

https://github.com/lindomarbatistao/raizes_do_nordeste.git

Todo o código-fonte desenvolvido encontra-se versionado utilizando Git e hospedado no GitHub, permitindo controle de versões e rastreabilidade das alterações realizadas durante o desenvolvimento.

4.5 Evidências da Implementação

4.5.1 – Back-end

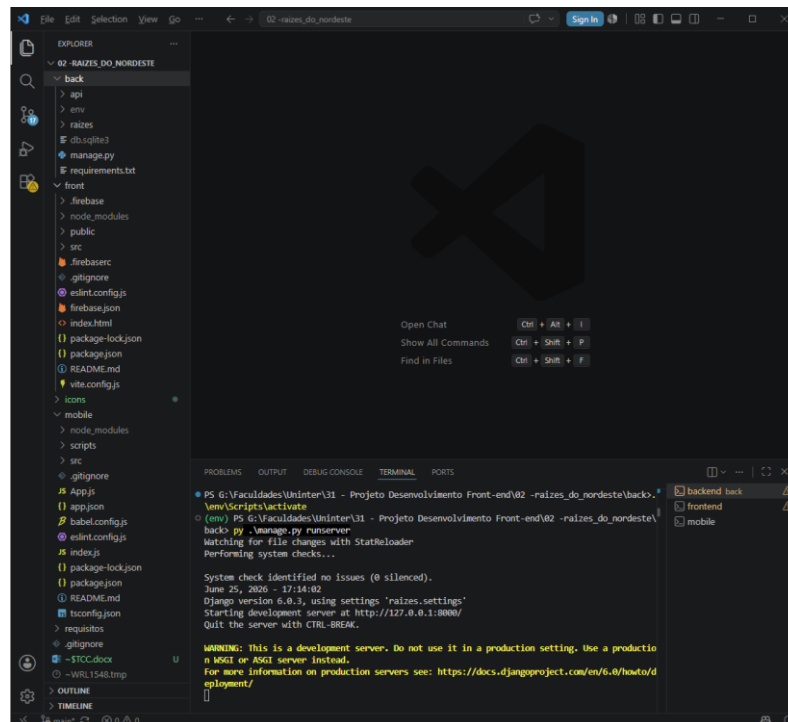


Figura 6 - Back-end em funcionamento

Fonte: Elaborado pelo autor (2026).

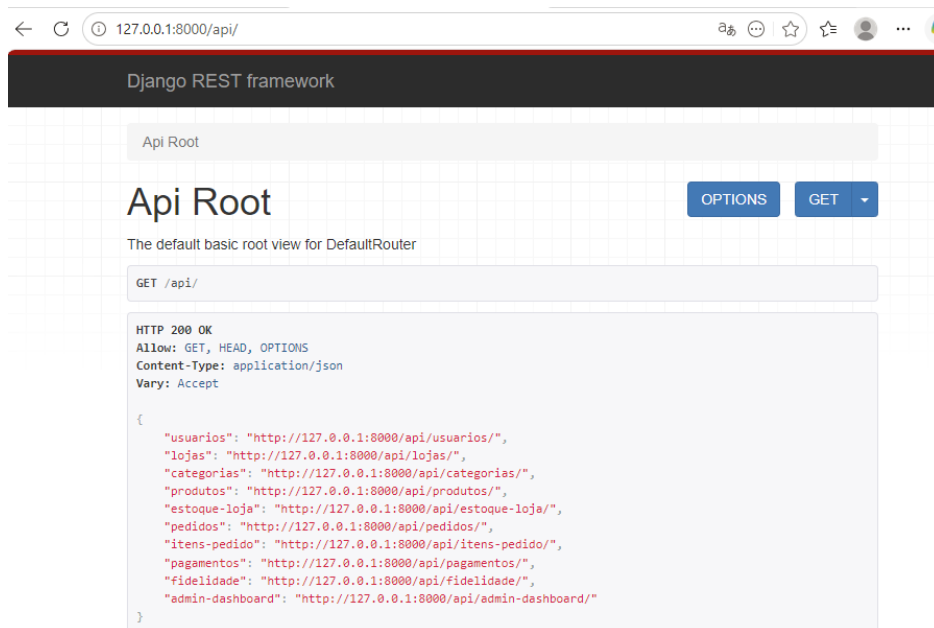


Figura 7 - Back-end em funcionamento no browser

Fonte: Elaborado pelo autor (2026).

5 PLANO DE TESTES E EVIDÊNCIAS DO BACKEND

5.1 Estratégia de Validação

A validação do sistema foi realizada por meio de testes funcionais utilizando a ferramenta Insomnia. Os testes tiveram como objetivo verificar o correto funcionamento dos endpoints da API, a autenticação dos usuários, a persistência dos dados no banco de dados e a aplicação das regras de negócio definidas nos requisitos do sistema.

Foram executados cenários positivos e negativos, simulando situações reais de uso da aplicação, permitindo validar tanto o comportamento esperado quanto o tratamento de erros.

5.2 Cadastro Inicial de Usuários

Antes da população das demais tabelas do banco de dados, foi necessário criar usuários para simular clientes reais da plataforma. Para essa etapa foi utilizado o shell interativo do Django, acessado por meio do comando:

```
python manage.py shell
```

A criação dos usuários foi realizada utilizando o método `create_user()`, disponibilizado pelo modelo de autenticação do framework. Essa abordagem foi escolhida porque o Django realiza automaticamente o processamento e a criptografia das senhas utilizando algoritmos seguros, garantindo que as credenciais não sejam armazenadas em texto puro no banco de dados.

Foram cadastrados dez usuários do tipo **CLIENTE** (**cliente01**, **cliente02**, **cliente03** etc), contendo informações como nome de usuário, nome, sobrenome, e-mail, telefone e CPF. Todos os usuários receberam inicialmente a senha padrão "**123456**", utilizada exclusivamente para fins de desenvolvimento e testes da aplicação.

O código utilizado para a geração automática dos registros foi desenvolvido em Python e executado diretamente no ambiente Django, conforme apresentado a seguir:

Evidência de Execução

```
(env) PS C:\Users\desktop\Desktop\Uninter\raizes_do_nordeste\back> python manage.py shell
Traceback (most recent call last):
  File "<console>", line 1, in <module>
ModuleNotFoundError: No module named 'usuarios'
>>>
>>> usuarios = [
...     ("cliente01", "João", "Silva", "cliente01@email.com", "19999990001", "11311111911"),
...     ("cliente02", "Jose", "Silva", "cliente01@email.com", "19999990001", "11311911111"),
...     ("cliente03", "Pedro", "Silva", "cliente01@email.com", "19999990001", "11311111111"),
...     ("cliente04", "Maria", "Souza", "cliente02@email.com", "19999990002", "22222222222"),
...     ("cliente05", "Bruna", "Silva", "cliente01@email.com", "19999990001", "11111611111"),
...     ("cliente06", "Matheus", "Souza", "cliente02@email.com", "19999990002", "22222262222"),
...     ("cliente07", "Carlos", "Silva", "cliente01@email.com", "19999990001", "16111111111"),
...     ("cliente08", "Giavana", "Souza", "cliente02@email.com", "19999990002", "22226222222"),
...     ("cliente09", "Gabriel", "Silva", "cliente01@email.com", "19999990001", "11111111161"),
...     ("cliente10", "Thomaz", "Souza", "cliente02@email.com", "19999990002", "22262222222"),
... ]
>>>
>>> for username, first_name, last_name, email, telefone, cpf in usuarios:
...     Usuario.objects.create_user(
...         username=username,
...         password="123456",
...         first_name=first_name,
...         last_name=last_name,
...         email=email,
...         telefone=telefone,
...         cpf=cpf,
...         tipo="CLIENTE"
...     )
...
<Usuario: cliente01>
<Usuario: cliente02>
<Usuario: cliente03>
<Usuario: cliente04>
<Usuario: cliente05>
<Usuario: cliente06>
<Usuario: cliente07>
<Usuario: cliente08>
<Usuario: cliente09>
<Usuario: cliente10>
```

Figura 8 - Criando usuários.

Fonte: Elaborado pelo autor (2026).

Evidência de Execução

	id	password	last_login	is_superuser	username	first_name	last_name	email	is_staff	is_active	date_joined	tipo	telefone	cpf
1	1	pbkdf2_sha...	NULL	0	cliente01	João	Silva	cliente01@email.com	0	1	2026-06-...	CLIENTE	19999990001	11311111911
2	2	pbkdf2_sha...	NULL	0	cliente02	Jose	Silva	cliente01@email.com	0	1	2026-06-...	CLIENTE	19999990001	11311911111
3	3	pbkdf2_sha...	NULL	0	cliente03	Pedro	Silva	cliente01@email.com	0	1	2026-06-...	CLIENTE	19999990001	11311111111
4	4	pbkdf2_sha...	NULL	0	cliente04	Maria	Souza	cliente02@email.com	0	1	2026-06-...	CLIENTE	19999990002	22222222222
5	5	pbkdf2_sha...	NULL	0	cliente05	Bruna	Silva	cliente01@email.com	0	1	2026-06-...	CLIENTE	19999990001	11111611111
6	6	pbkdf2_sha...	NULL	0	cliente06	Matheus	Souza	cliente02@email.com	0	1	2026-06-...	CLIENTE	19999990002	22222262222
7	7	pbkdf2_sha...	NULL	0	cliente07	Carlos	Silva	cliente01@email.com	0	1	2026-06-...	CLIENTE	19999990001	16111111111
8	8	pbkdf2_sha...	NULL	0	cliente08	Giavana	Souza	cliente02@email.com	0	1	2026-06-...	CLIENTE	19999990002	22226222222
9	9	pbkdf2_sha...	NULL	0	cliente09	Gabriel	Silva	cliente01@email.com	0	1	2026-06-...	CLIENTE	19999990001	11111111161
10	10	pbkdf2_sha...	NULL	0	cliente10	Thomaz	Souza	cliente02@email.com	0	1	2026-06-...	CLIENTE	19999990002	22262222222

Figura 9 - Tabela de usuários já populada.

Fonte: Elaborado pelo autor (2026).

Após a execução do script, os registros foram armazenados na tabela de usuários do sistema e passaram a ser utilizados nos testes de autenticação, criação de pedidos, programa de fidelidade e demais funcionalidades relacionadas aos clientes.

5.3 População Inicial do Banco de Dados

Após a criação da estrutura do banco de dados por meio das migrações do Django e do cadastro inicial dos usuários utilizando o ORM¹ da aplicação, foi realizada a carga inicial das demais tabelas por meio de comandos SQL executados diretamente no banco de dados SQLite. Essa etapa teve como objetivo disponibilizar uma base de dados consistente para validação das funcionalidades desenvolvidas, possibilitando a execução de testes completos envolvendo o gerenciamento de lojas, produtos, categorias, estoque, pedidos, pagamentos e programa de fidelidade. A utilização de dados previamente cadastrados permitiu simular cenários reais de operação da rede Raízes do Nordeste, contribuindo para a verificação das regras de negócio e da integração entre os diferentes módulos do sistema. Inicialmente foram cadastradas dez unidades da rede Raízes do Nordeste, distribuídas entre os municípios de Campinas, Sumaré e Hortolândia. Cada registro contém informações de identificação da unidade, endereço, cidade, estado e situação de funcionamento.

¹ ORM (Object-Relational Mapping) é uma técnica de desenvolvimento que permite o mapeamento entre objetos da aplicação e tabelas do banco de dados, possibilitando a manipulação dos dados sem a necessidade de escrever comandos SQL diretamente.

Em seguida, foram cadastradas dez categorias de produtos, representando os principais grupos comercializados pela rede, como cuscuz, tapiocas, bebidas, sobremesas, pratos executivos, lanches, cafés, sucos, combos e salgados.

Posteriormente foram inseridos dez produtos de exemplo, contendo informações como nome, descrição, preço, categoria e situação de disponibilidade. Esses produtos simulam itens reais do cardápio da empresa e serviram como base para os testes de consulta e criação de pedidos.

Para possibilitar o gerenciamento de estoque por unidade, foi realizada a população da tabela de estoque, associando cada produto a cada loja cadastrada. Foram definidos valores iniciais de quantidade disponível, permitindo validar consultas e futuras operações de controle de estoque.

Também foram inseridos registros de pedidos simulando compras realizadas por diferentes clientes e por diferentes canais de atendimento, incluindo aplicação web, aplicativo mobile, balcão de atendimento e totens de autoatendimento. Cada pedido recebeu um status representando diferentes etapas do fluxo operacional, tais como "CRIADO", "PAGO", "EM_PREPARO", "PRONTO" e "ENTREGUE".

Após a criação dos pedidos, foram cadastrados os respectivos itens de pedido, estabelecendo o relacionamento entre pedidos e produtos e permitindo o cálculo dos valores totais das compras realizadas.

A população do banco também contemplou registros de pagamentos, utilizando diferentes métodos como PIX, cartão e dinheiro. Foram simulados pagamentos aprovados e pendentes, possibilitando validar cenários distintos durante os testes da aplicação.

Por fim, foi realizada a carga inicial da tabela de fidelidade, associando pontuações aos clientes cadastrados. Essa estrutura foi criada visando futuras implementações relacionadas ao programa de fidelização da rede Raízes do Nordeste.

A utilização de dados previamente cadastrados permitiu validar todas as funcionalidades principais do sistema, além de facilitar a execução dos testes descritos no Plano de Testes e Evidências.

Evidência de Execução

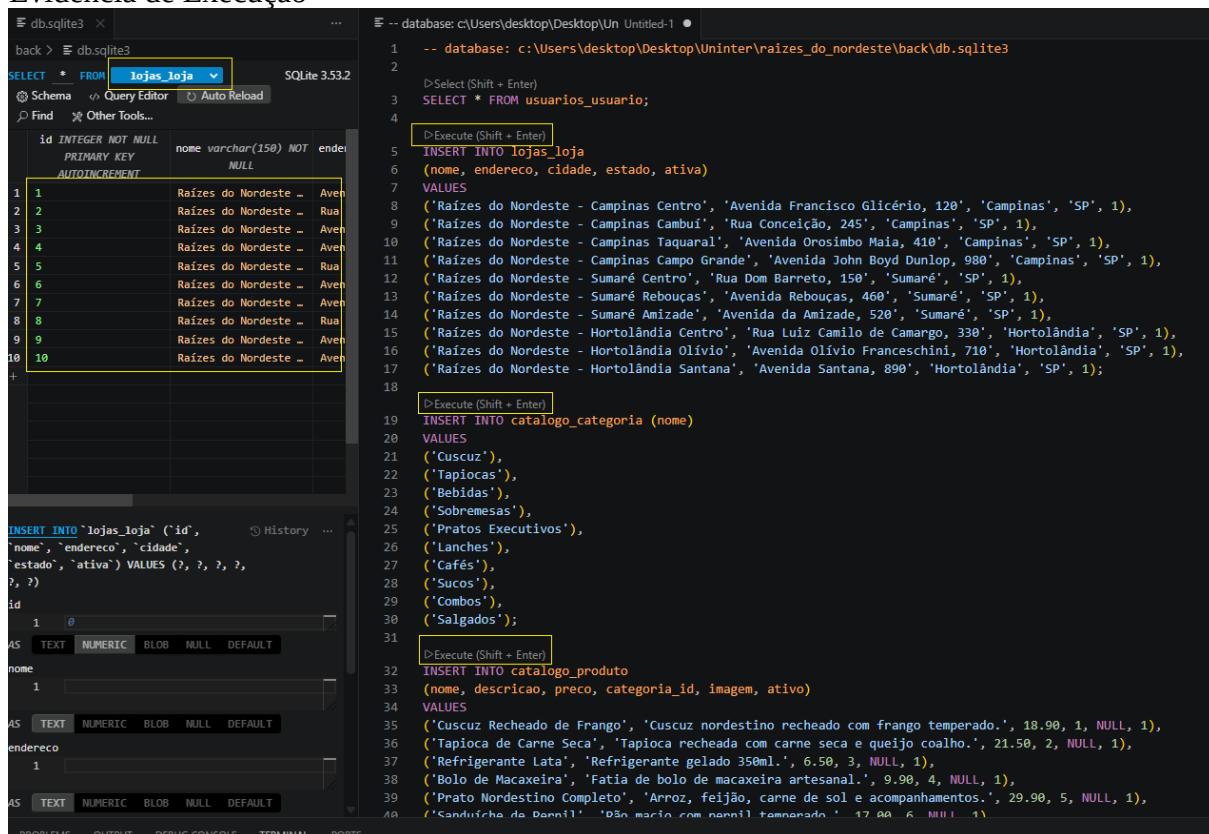


Figura 10 - Populando todas as tabelas do banco.

Fonte: Elaborado pelo autor (2026).

5.4 Listagem de usuários

A Figura 37 apresenta a execução do endpoint de listagem de usuários por meio da ferramenta Insomnia. Para a realização do teste foi utilizada uma requisição HTTP GET para a rota /api/usuarios/, enviando um token de autenticação JWT no cabeçalho da requisição. Como resultado, o sistema retornou o código de sucesso HTTP 200 (OK) e uma lista contendo os usuários cadastrados no banco de dados, incluindo informações como nome de usuário, nome, sobrenome, e-mail, telefone, CPF e tipo de usuário. Esse teste permitiu validar o correto funcionamento da API, a autenticação do usuário e a comunicação entre a aplicação e o banco de dados.

Evidência de Execução

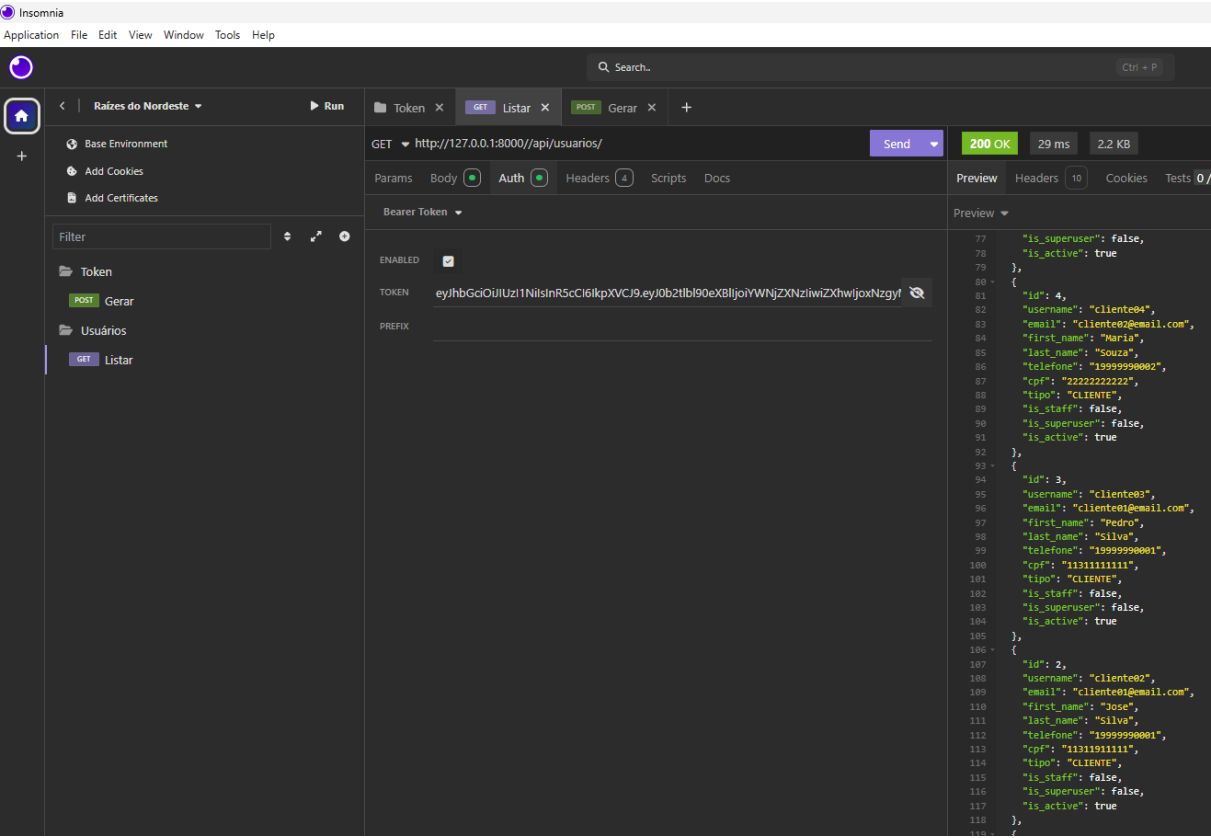


Figura 11 - Listagem de usuários utilizando o Insomnia.

Fonte: Elaborado pelo autor (2026).

5.5 Cadastro de Usuário

A Figura 38 apresenta o teste de cadastro de usuários realizado por meio da ferramenta Insomnia. Para a execução do teste foi enviada uma requisição HTTP POST para o endpoint /api/usuarios/, informando os dados necessários para a criação de um novo usuário no formato JSON.

Como pode ser observado, foi realizado o cadastro do usuário **cliente50**, informando nome de usuário, senha e endereço de e-mail. Após o envio da requisição, a API retornou o código HTTP **201 Created**, indicando que o registro foi criado com sucesso no banco de dados.

Na resposta da requisição, o sistema retornou os dados do usuário recém-cadastrado, incluindo identificador único (ID), nome de usuário, e-mail, tipo de usuário e demais informações associadas ao cadastro. Esse teste permitiu validar o correto funcionamento do endpoint responsável pela criação de usuários e a integração entre a API e o banco de dados.

Evidência de Execução

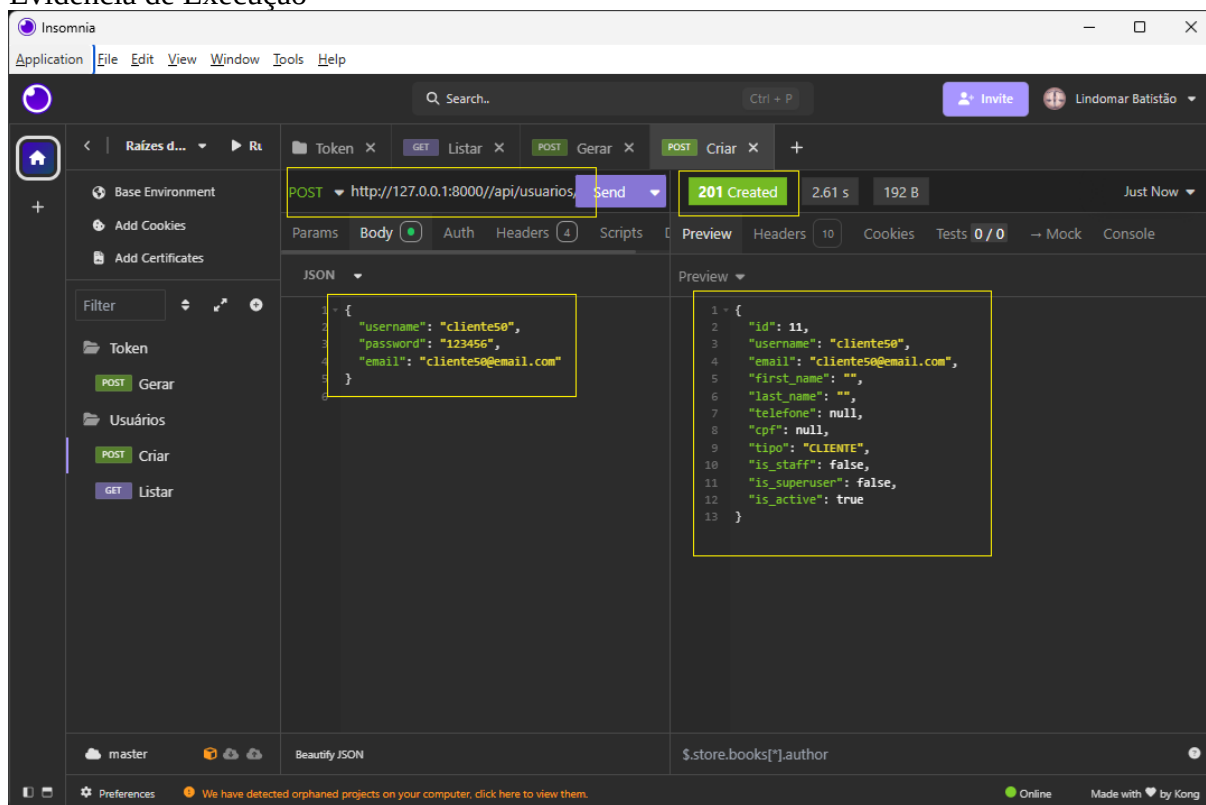


Figura 12 - Cadastro de usuário utilizando o endpoint de criação da API.

Fonte: Elaborado pelo autor (2026).

5.6 Autenticação do Usuário

A Figura 39 apresenta a obtenção do token de autenticação da API utilizando a ferramenta Insomnia. Para realizar esse procedimento, foi enviada uma requisição HTTP POST para o endpoint `/api/token/`, informando as credenciais de acesso do usuário previamente cadastrado no sistema.

Como resultado, a API retornou o código HTTP 200 (OK), indicando que a autenticação foi realizada com sucesso. Além disso, foram gerados os tokens access e refresh, utilizados pelo mecanismo de autenticação JWT (JSON Web Token) implementado no sistema.

Como pode ser observado na figura, o token foi obtido a partir do login do usuário e passou a ser utilizado nas demais requisições protegidas da API. Esse procedimento garante que apenas usuários autenticados possam acessar recursos restritos, como consulta de pedidos, informações do perfil e demais funcionalidades vinculadas ao cliente.

[illegible]

Fonte: Elaborado pelo autor (2026).

A Figura 40 apresenta o teste realizado no endpoint responsável por retornar os dados do usuário autenticado. Para a execução desse teste foi utilizada uma requisição HTTP GET para a rota `/api/usuarios/me/`, enviando no cabeçalho da requisição um token JWT válido previamente obtido durante o processo de autenticação.

Como pode ser observado, a API retornou o código HTTP **200 OK**, indicando que a autenticação foi validada com sucesso. Na resposta foram exibidas as informações do usuário autenticado, incluindo identificador, nome de usuário, e-mail, nome, sobrenome, telefone, CPF, tipo de usuário e demais atributos cadastrados no sistema.

Esse teste foi fundamental para validar o funcionamento do mecanismo de autenticação baseado em JWT, garantindo que apenas usuários devidamente autenticados possam acessar informações pessoais e recursos protegidos da aplicação.

Evidência de Execução

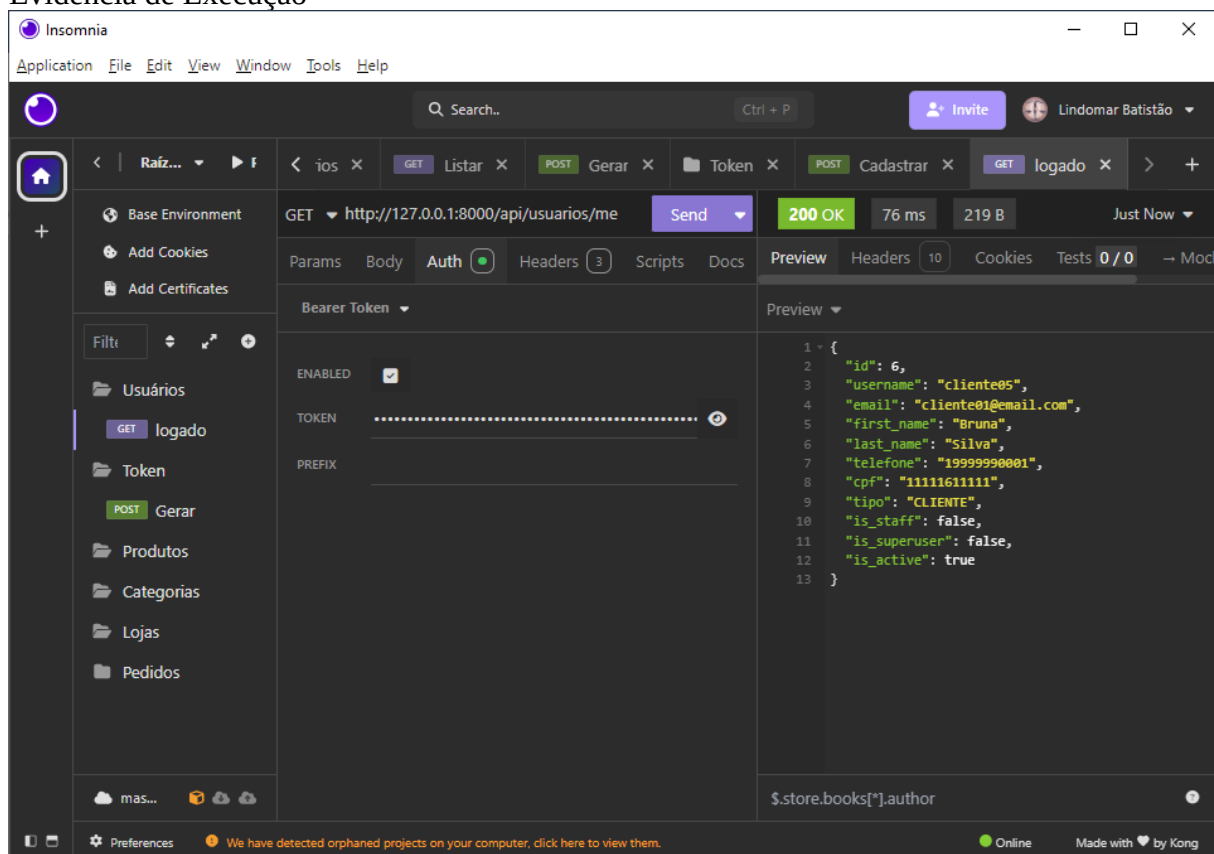


Figura 14 - Consulta dos dados do usuário autenticado por meio do endpoint /api/usuarios/me/.

Fonte: Elaborado pelo autor (2026).

5.8 Listagem de Produtos

A Figura 41 apresenta o teste de listagem de produtos realizado por meio da ferramenta Insomnia. Para a execução desse teste foi utilizada uma requisição HTTP GET para o endpoint /api/produtos/, responsável por retornar todos os produtos cadastrados no sistema.

Como resultado, a API retornou o código HTTP **200 OK**, indicando que a consulta foi executada com sucesso. Na resposta da requisição é possível observar a lista de produtos disponíveis para comercialização, contendo informações como identificador, nome, descrição, preço, categoria, imagem e status de disponibilidade.

A validação desse endpoint é fundamental para garantir o correto funcionamento das funcionalidades de consulta de produtos tanto na aplicação web quanto no aplicativo móvel, uma vez que essas interfaces dependem dessas informações para exibir o catálogo disponível aos clientes.

Evidência de Execução

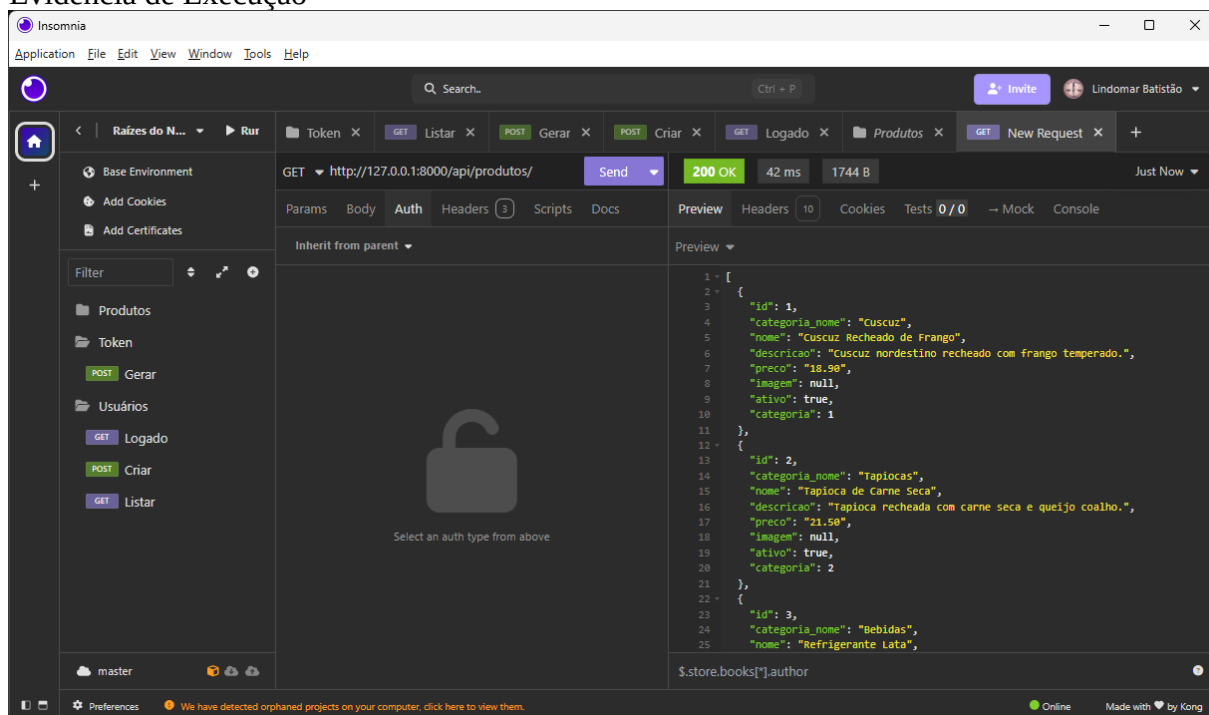


Figura 15 - Listagem de produtos por meio do endpoint /api/produtos/.

Fonte: Elaborado pelo autor (2026).

5.9 Listagem de Categorias

A Figura 42 apresenta o teste de listagem das categorias cadastradas no sistema por meio da ferramenta Insomnia. Para a realização desse teste foi utilizada uma requisição HTTP GET para o endpoint /api/categorias/, responsável por disponibilizar as categorias utilizadas na organização dos produtos comercializados pela rede Raízes do Nordeste.

Como resultado, a API retornou o código HTTP **200 OK**, confirmando que a requisição foi processada com sucesso. Na resposta é possível visualizar as categorias cadastradas no banco de dados, incluindo informações como identificador e nome da categoria. Entre as categorias retornadas encontram-se Cuscuz, Tapiocas, Bebidas, Sobremesas, Pratos Executivos, Lanches, Cafés, Sucos, Combos e Salgados.

A validação desse endpoint é importante para garantir a correta classificação dos produtos no sistema, permitindo que as aplicações cliente apresentem os itens organizados de forma adequada, melhorando a navegação e a experiência do usuário.

Evidência de Execução

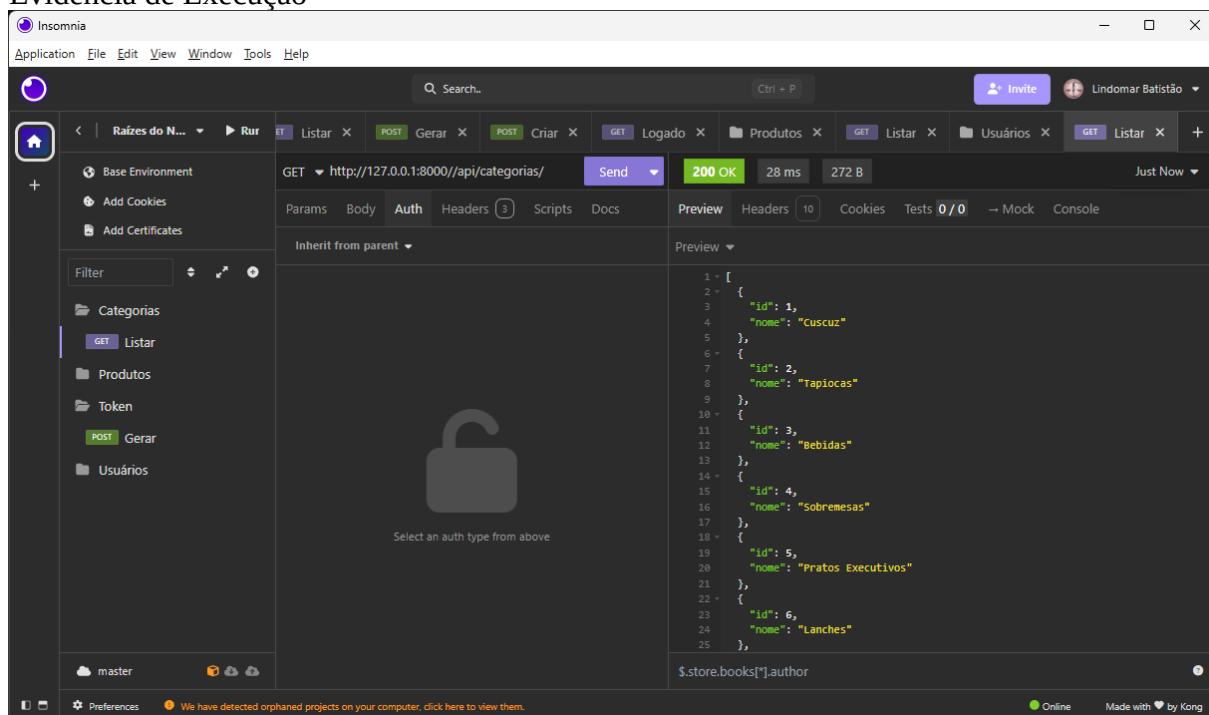


Figura 16 - Listagem das categorias cadastradas por meio do endpoint /api/categorias/.

Fonte: Elaborado pelo autor (2026).

5.10 Listagem de Lojas

A Figura 43 apresenta o teste de listagem das lojas cadastradas na aplicação utilizando a ferramenta Insomnia. Para a execução do teste foi realizada uma requisição HTTP GET para o endpoint /api/lojas/, responsável por retornar todas as unidades da rede Raízes do Nordeste registradas no banco de dados.

Como resultado, a API retornou o código HTTP **200 OK**, indicando que a consulta foi executada com sucesso. Na resposta da requisição são exibidas informações detalhadas das lojas cadastradas, incluindo identificador, nome da unidade, endereço, cidade, estado e situação de funcionamento.

A validação desse endpoint é importante para garantir que as aplicações cliente possam consultar corretamente as unidades disponíveis, permitindo ao usuário selecionar a loja desejada durante o processo de compra. Além disso, essa funcionalidade é utilizada para vincular pedidos, estoques e demais operações às respectivas unidades da rede.

Evidência de Execução

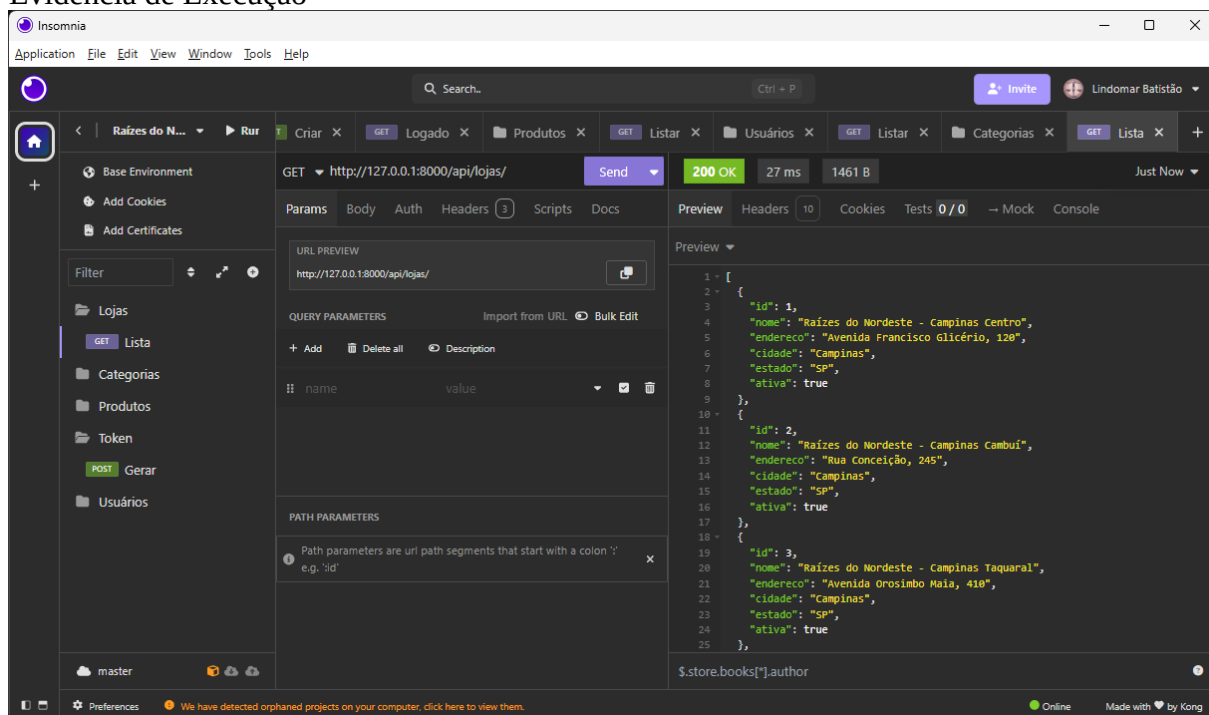


Figura 17 - – Listagem das lojas cadastradas por meio do endpoint `/api/lojas/`.

Fonte: Elaborado pelo autor (2026).

5.11 Criação de Pedido

A Figura 44 apresenta o teste de criação de pedidos realizado por meio da ferramenta Insomnia. Para a execução desse teste foi utilizada uma requisição HTTP POST para o endpoint `/api/pedidos/`, responsável por registrar novos pedidos no sistema.

Como pode ser observado, foi enviado um objeto JSON contendo a loja selecionada, o canal de venda utilizado e os itens que compõem o pedido. Nesse cenário, foi solicitado um pedido para a loja de identificador 1, utilizando o canal MOBILE, contendo uma unidade do produto "Tapioca de Carne Seca".

Como resultado, a API retornou o código HTTP **201 Created**, indicando que o pedido foi criado com sucesso. Na resposta da requisição são exibidas informações detalhadas do pedido gerado, incluindo identificador do pedido, cliente associado, loja selecionada, canal de venda, status atual, valor total da compra, datas de criação e atualização, além dos itens registrados no pedido.

A validação desse endpoint é fundamental para garantir o correto funcionamento do processo de vendas da aplicação, permitindo que pedidos realizados pelos clientes sejam armazenados e processados adequadamente pelo sistema.

Evidência de Execução

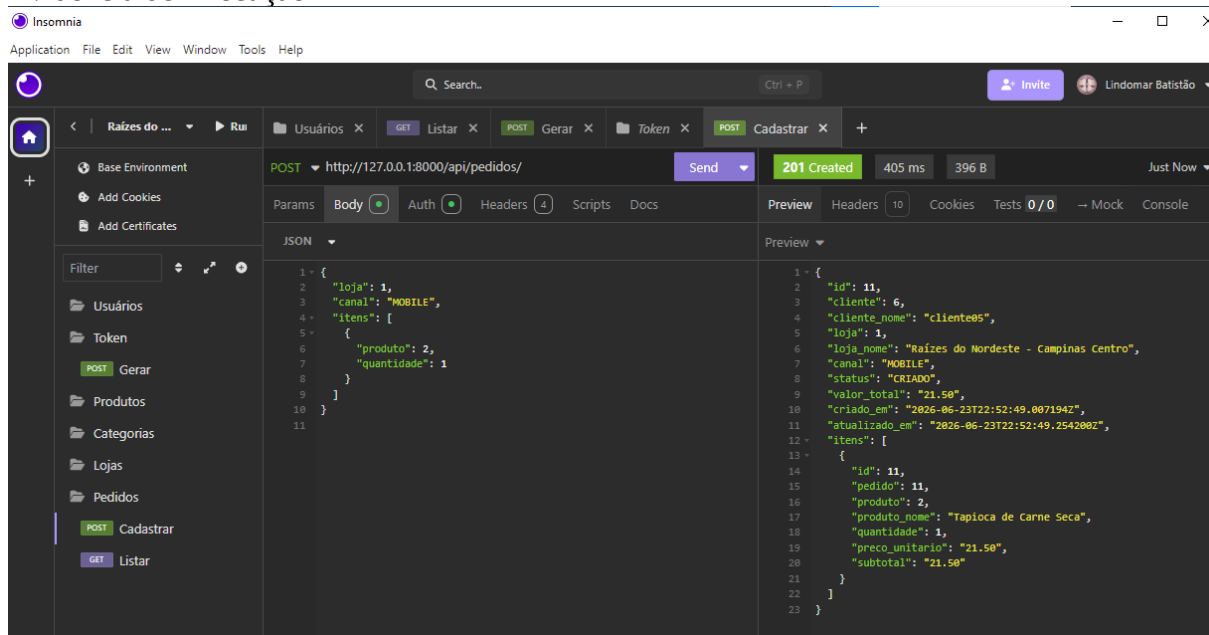


Figura 18 - Criação de pedido por meio do endpoint /api/pedidos/.

Fonte: Elaborado pelo autor (2026).

5.12 Consulta dos Pedidos

A Figura 45 apresenta o teste de consulta dos pedidos realizados por meio da ferramenta Insomnia. Para a execução desse teste foi utilizada uma requisição HTTP GET para o endpoint /api/pedidos/, responsável por retornar os pedidos vinculados ao usuário autenticado no sistema.

Como resultado, a API retornou o código HTTP **200 OK**, indicando que a consulta foi processada com sucesso. Na resposta da requisição é possível visualizar a lista de pedidos cadastrados, contendo informações como identificador do pedido, cliente associado, loja responsável pelo atendimento, canal de venda utilizado, status do pedido, valor total da compra, datas de criação e atualização, além dos itens que compõem cada pedido.

A validação desse endpoint é importante para garantir que os usuários possam acompanhar corretamente seus pedidos e que os administradores tenham acesso às informações necessárias para o gerenciamento das vendas realizadas pela plataforma.

Evidência de Execução

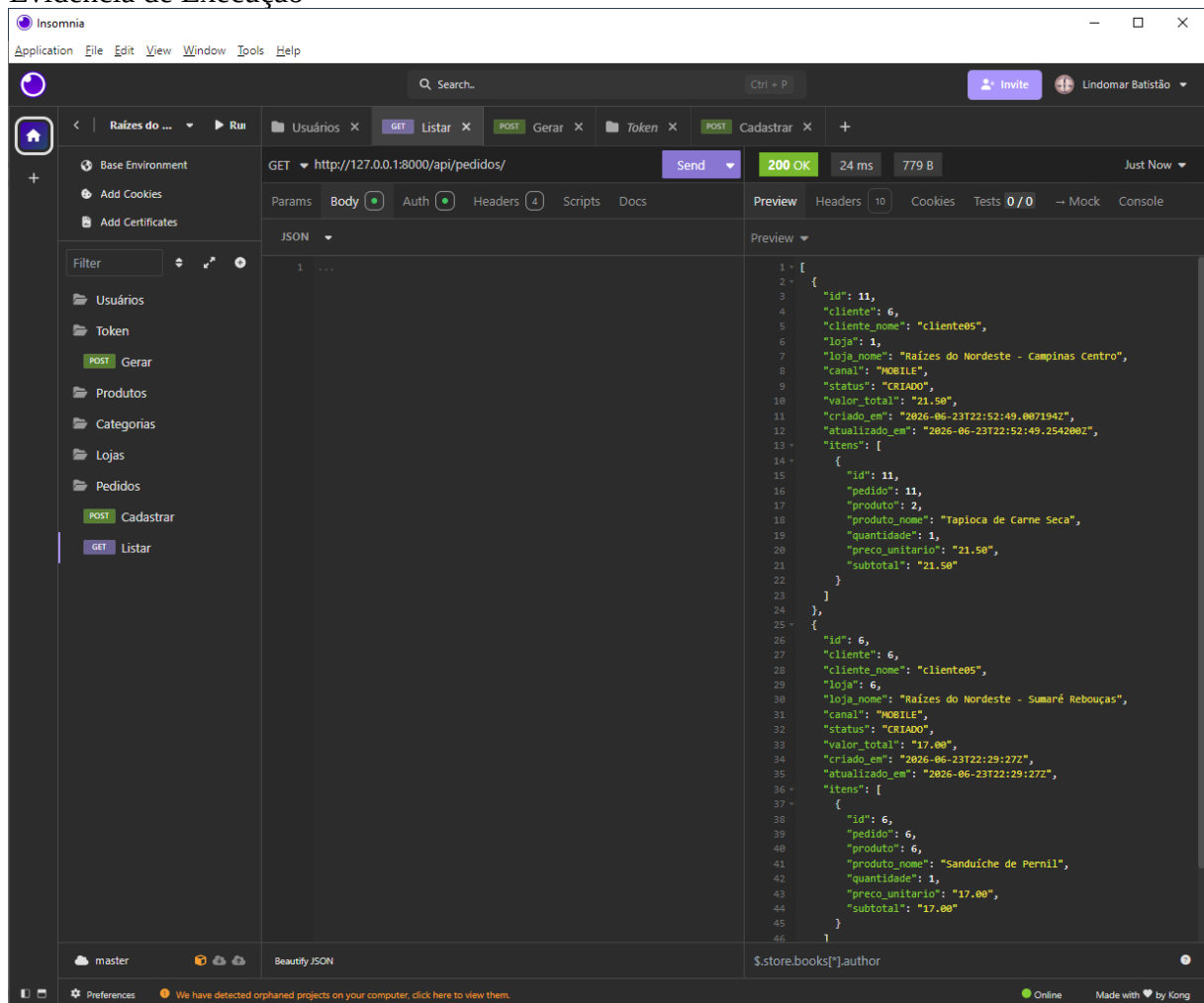


Figura 19 - Consulta dos pedidos por meio do endpoint `/api/pedidos/`.

Fonte: Elaborado pelo autor (2026).

5.13 Acesso sem Autenticação (Teste Negativo)

A Figura 46 apresenta um teste negativo realizado com o objetivo de validar a segurança das rotas protegidas da API. Para a execução desse teste foi utilizada uma requisição HTTP GET para o endpoint `/api/pedidos/`, porém sem o envio de um token de autenticação válido.

Como resultado, a API retornou o código HTTP **401 Unauthorized**, indicando que o acesso foi corretamente bloqueado por falta de credenciais de autenticação. Na resposta da requisição é apresentada a mensagem *"Authentication credentials were not provided."*, informando que o usuário não possui autorização para acessar o recurso solicitado.

A validação desse cenário é fundamental para garantir a segurança da aplicação, impedindo que usuários não autenticados tenham acesso a informações sensíveis, como dados de pedidos e informações de clientes. Dessa forma, o teste comprova que o mecanismo de

autenticação implementado está funcionando corretamente e protegendo as rotas restritas do sistema.

Evidência de Execução

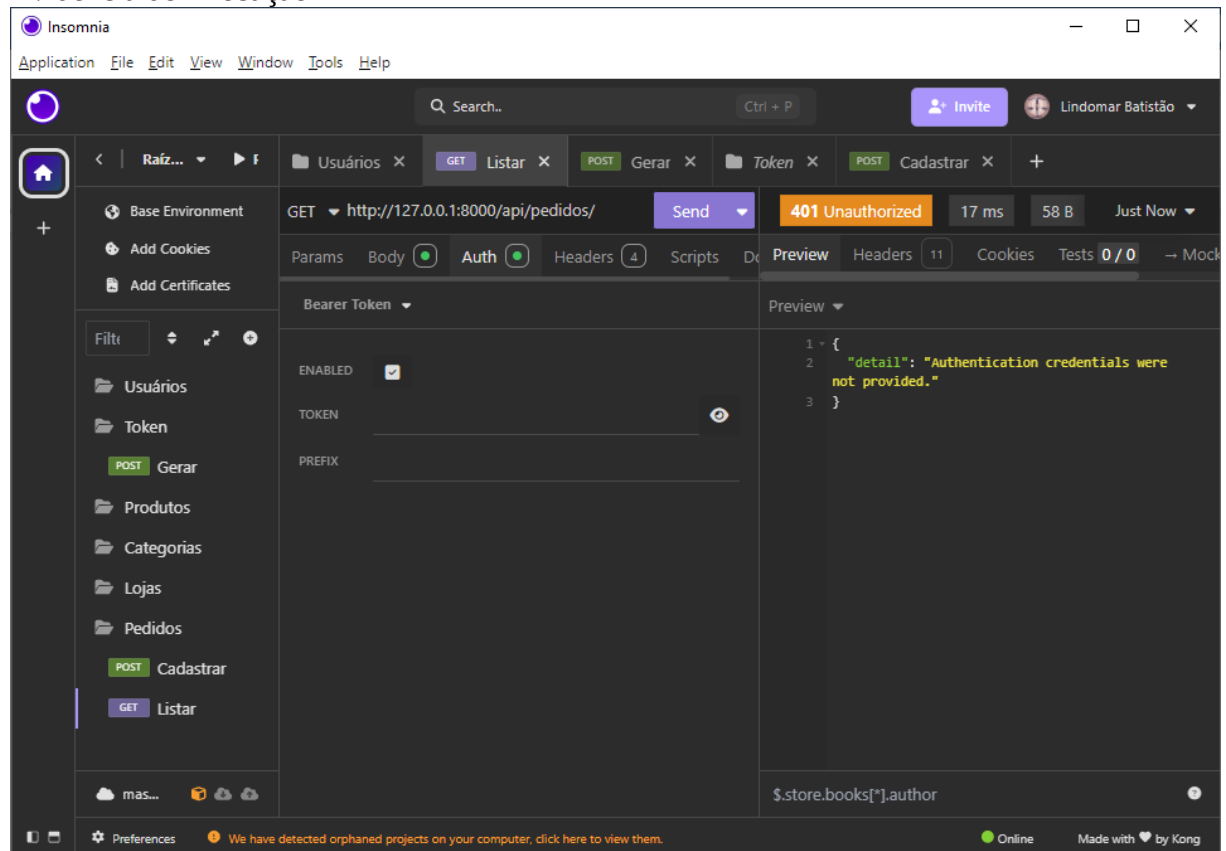


Figura 20 - Tentativa de acesso ao endpoint /api/pedidos/ sem autenticação.

Fonte: Elaborado pelo autor (2026).

5.14 Criação de Pedido sem Loja (Teste Negativo)

A Figura 47 apresenta um teste negativo realizado com o objetivo de validar as regras de negócio e a obrigatoriedade dos campos necessários para a criação de um pedido. Para a execução desse teste foi utilizada uma requisição HTTP POST para o endpoint /api/pedidos/, porém sem informar o campo **loja**, que é obrigatório para a associação do pedido a uma unidade da rede.

Como resultado, a API retornou o código HTTP **400 Bad Request**, indicando que os dados enviados não atenderam às validações definidas pelo sistema. Na resposta da requisição é apresentada a mensagem "**This field is required.**", vinculada ao atributo **loja**, informando que o campo não foi fornecido na solicitação.

A validação desse cenário é importante para garantir a integridade dos dados armazenados no sistema, impedindo que pedidos sejam criados sem uma loja associada. Dessa

forma, o teste comprova que as validações implementadas na API estão funcionando corretamente e impedem o cadastro de informações incompletas ou inconsistentes.

Evidência de Execução

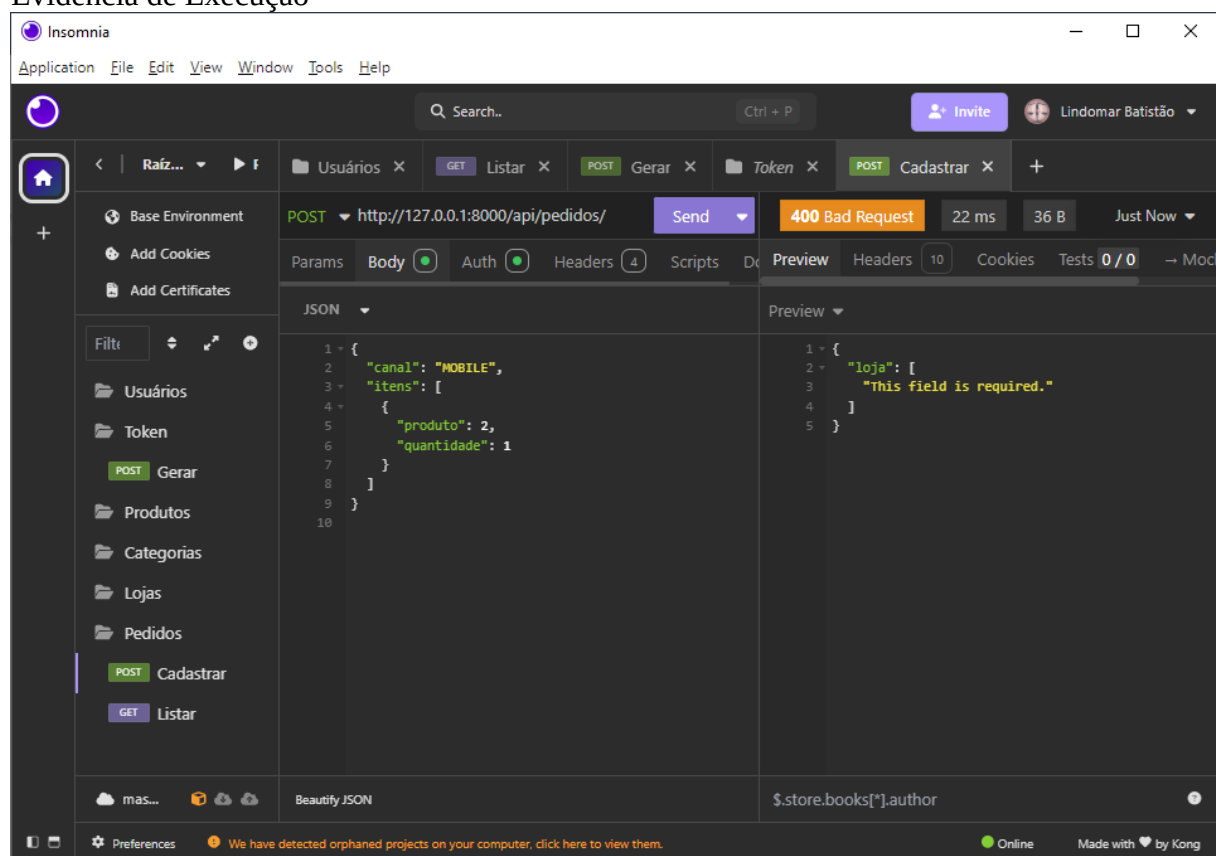


Figura 21 - Tentativa de criação de pedido sem informar o campo obrigatório "loja".

Fonte: Elaborado pelo autor (2026).

5.15 Geração de token em ambiente hospedado na nuvem

A Figura 48 apresenta a autenticação da API utilizando uma URL hospedada em **ambiente de nuvem**, permitindo que os serviços sejam acessados pela Internet sem a necessidade de execução local do servidor. Para esse teste foi utilizado o endpoint `/api/token/`, responsável por validar as credenciais do usuário e retornar os tokens de autenticação no padrão **JWT (JSON Web Token)**.

A utilização de uma infraestrutura hospedada demonstra que a aplicação está preparada para operar em um ambiente semelhante ao de produção, possibilitando o acesso remoto por diferentes clientes, como aplicações Web e dispositivos móveis. Após a validação das credenciais, a API retorna os tokens **access** e **refresh**, que serão utilizados para autenticar as demais requisições realizadas pelo sistema.

A disponibilização da API em um servidor hospedado na nuvem permite maior disponibilidade do sistema, eliminando a dependência de um ambiente local de desenvolvimento. Dessa forma, clientes autenticados podem acessar os serviços de qualquer local com conexão à Internet, aproximando a solução de um cenário real de utilização em produção.

Com o objetivo de aproximar o projeto de um cenário real de produção, a API REST foi publicada na plataforma **PythonAnywhere**².

A utilização dessa plataforma permitiu que a API fosse acessada remotamente, dispensando a execução do servidor de desenvolvimento na máquina local. Dessa forma, tanto a aplicação Web quanto o aplicativo Mobile puderam consumir os serviços da API utilizando uma URL pública, simulando o funcionamento de um sistema em ambiente de produção.

O acesso à API hospedada foi realizado por meio do endereço:

<https://lintelecom.pythonanywhere.com/api/>

A Figura 48 apresenta a obtenção do token de autenticação diretamente no servidor hospedado, demonstrando que a aplicação está disponível para acesso via Internet.

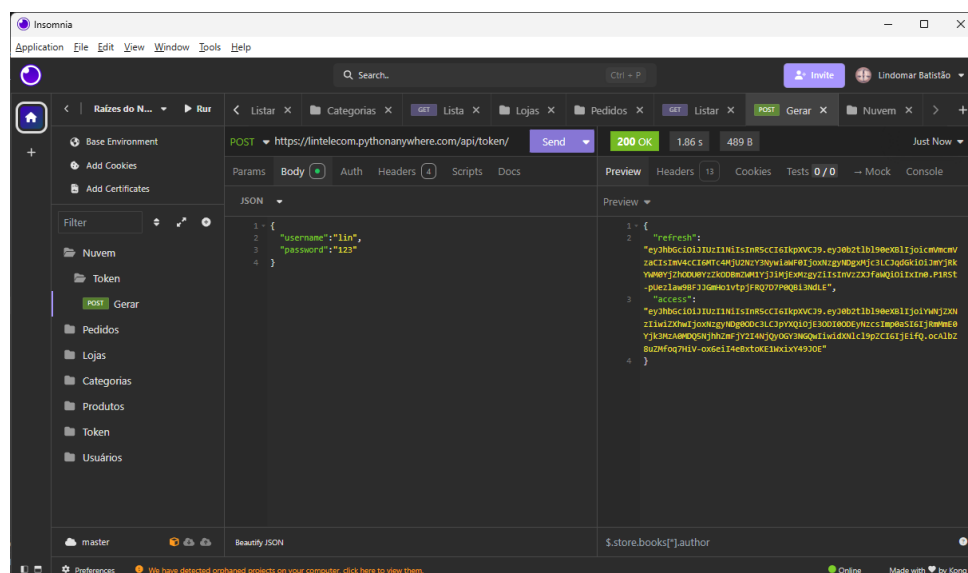


Figura 22 - Geração de token JWT utilizando a API hospedada em ambiente de nuvem.

Fonte: Elaborado pelo autor (2026).

² O PythonAnywhere ou www.pythonanywhere.com é um serviço de hospedagem em nuvem especializado em aplicações desenvolvidas em Python, oferecendo suporte nativo ao framework Django, banco de dados, gerenciamento do ambiente virtual e disponibilização da aplicação por meio de uma URL pública.

5.17 Considerações Finais dos Testes no Back-end

A execução dos testes realizados no back-end permitiu validar as principais funcionalidades desenvolvidas para o sistema Raízes do Nordeste. Por meio da ferramenta Insomnia, foi possível verificar o correto funcionamento dos endpoints responsáveis pela autenticação de usuários, consulta de informações cadastrais, gerenciamento de produtos, categorias, lojas e pedidos.

Os testes positivos demonstraram que a API é capaz de processar requisições válidas, retornando os códigos HTTP adequados e os dados esperados em formato JSON. Destacam-se as operações de autenticação, cadastro de usuários, consulta de produtos, listagem de categorias e lojas, além da criação e consulta de pedidos realizados pelos clientes.

Também foram executados testes negativos com o objetivo de validar as regras de segurança e integridade dos dados. Nesses cenários, o sistema respondeu corretamente ao bloquear acessos sem autenticação, retornando o código HTTP 401 (Unauthorized), e ao impedir o cadastro de pedidos com informações obrigatórias ausentes, retornando o código HTTP 400 (Bad Request). Esses resultados demonstram que as validações implementadas na API estão funcionando conforme o esperado.

De modo geral, os testes evidenciaram que a arquitetura desenvolvida atende aos requisitos propostos pelo estudo de caso, garantindo segurança, consistência dos dados e comunicação eficiente entre os diferentes canais de atendimento da empresa, como aplicativo mobile, sistema web, totens de autoatendimento e balcões de venda. Além disso, a utilização do framework Django REST Framework contribuiu para a construção de uma API organizada, escalável e preparada para futuras evoluções do sistema.

6 PLANO DE TESTES E EVIDÊNCIAS DO FRONTEND WEB

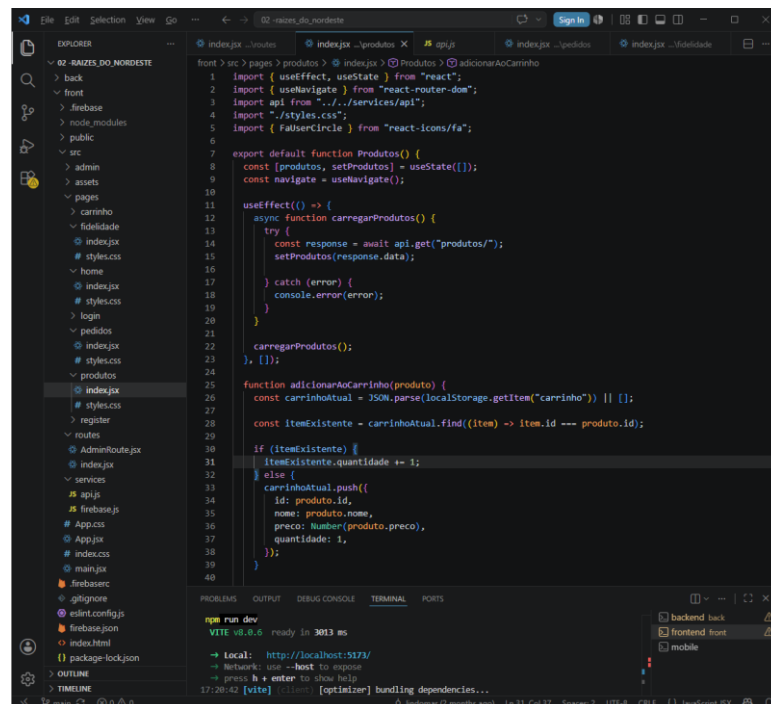


Figura 23 - Front-end em funcionamento

Fonte: Elaborado pelo autor (2026).

6.2 Login de usuário

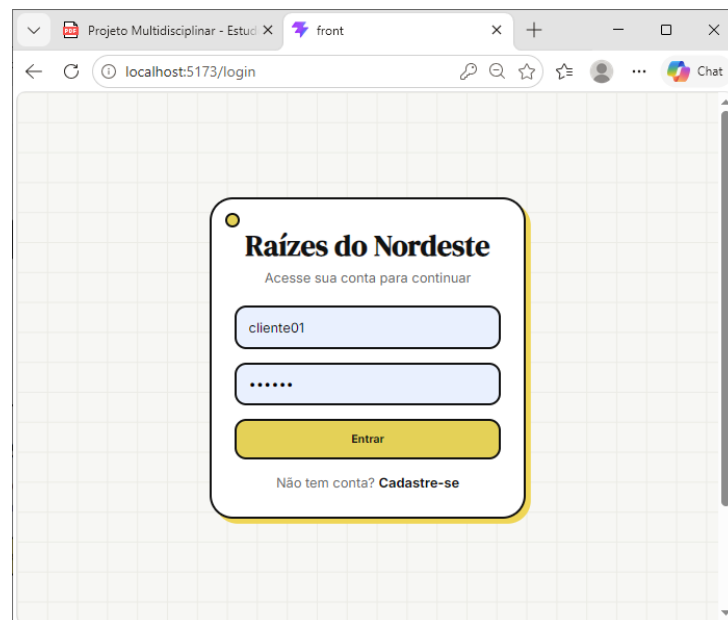


Figura 24 - Tela de login no front

Fonte: Elaborado pelo autor (2026).

6.3 Home com usuário logado



Figura 25 - Front-end em funcionamento no browser com usuário comum logado

Fonte: Elaborado pelo autor (2026).

6.4 Lista de Produtos



Figura 26 - Página de produtos

Fonte: Elaborado pelo autor (2026).

6.5 Pedidos



Figura 27 - Página de pedidos

Fonte: Elaborado pelo autor (2026).

6.6 Histórico de Pedidos



Figura 28 - Página de históricos de pedidos

Fonte: Elaborado pelo autor (2026).

6.7 Fidelidade



Figura 29 - Página do clube de fidelidade

Fonte: Elaborado pelo autor (2026).

6.8 Login de administrador



Figura 30 - Dashboard de administrador do sistema

Fonte: Elaborado pelo autor (2026).

6.9 Usuários

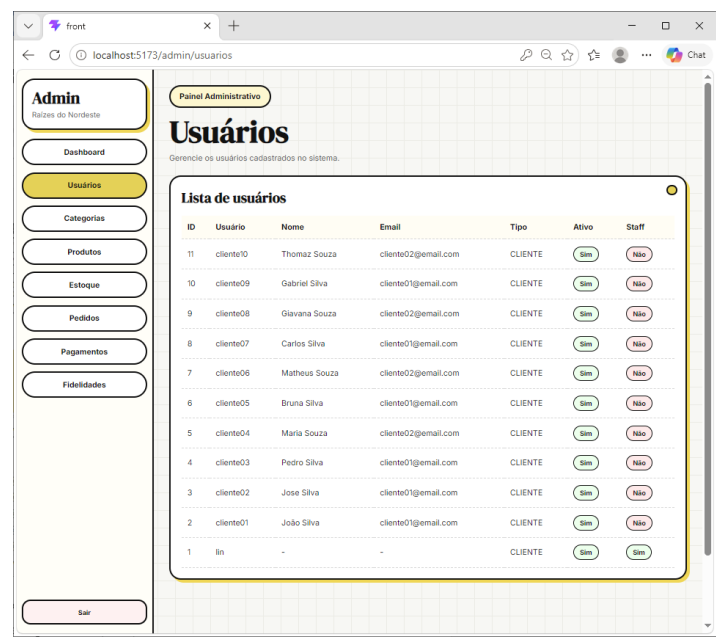


Figura 31 - Página de visualização dos usuários pelo administrador

Fonte: Elaborado pelo autor (2026).

6.10 Categorias

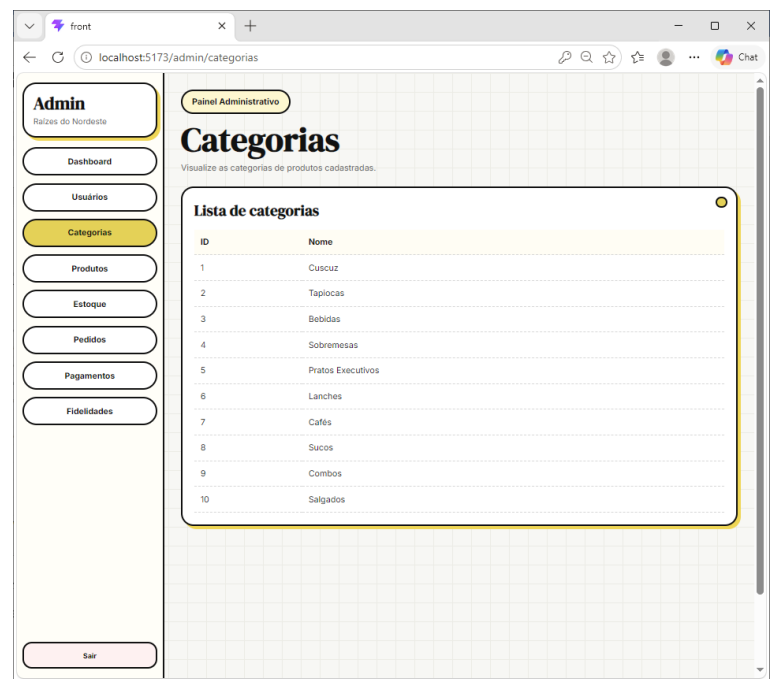
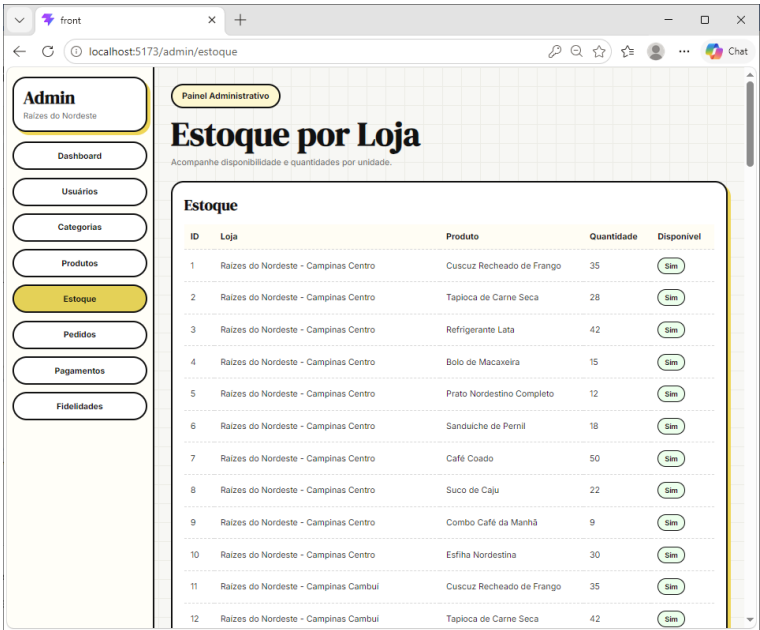


Figura 32 - Página de visualização de categorias pelo administrador

Fonte: Elaborado pelo autor (2026).

6.11 Estoque



Admin
Raízes do Nordeste

Painel Administrativo

Estoque por Loja

Acompanhe disponibilidade e quantidades por unidade.

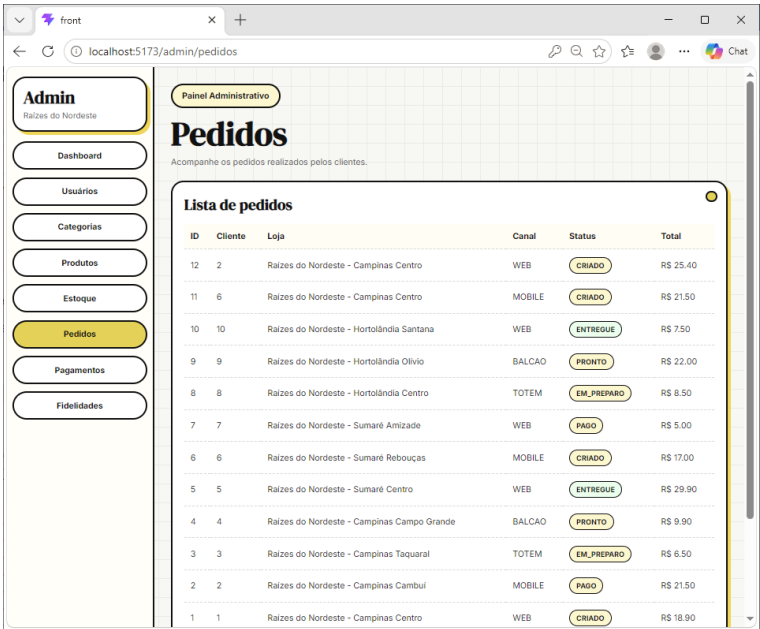
Estoque

ID	Loja	Produto	Quantidade	Disponível
1	Raízes do Nordeste - Campinas Centro	Cuscuz Recheado de Frango	35	<button>SIM</button>
2	Raízes do Nordeste - Campinas Centro	Tapioca de Carne Seca	28	<button>SIM</button>
3	Raízes do Nordeste - Campinas Centro	Refrigerante Lata	42	<button>SIM</button>
4	Raízes do Nordeste - Campinas Centro	Bolo de Macaxeira	15	<button>SIM</button>
5	Raízes do Nordeste - Campinas Centro	Prato Nordestino Completo	12	<button>SIM</button>
6	Raízes do Nordeste - Campinas Centro	Sanduíche de Pernil	18	<button>SIM</button>
7	Raízes do Nordeste - Campinas Centro	Café Coado	50	<button>SIM</button>
8	Raízes do Nordeste - Campinas Centro	Suco de Caju	22	<button>SIM</button>
9	Raízes do Nordeste - Campinas Centro	Combo Café da Manhã	9	<button>SIM</button>
10	Raízes do Nordeste - Campinas Centro	Esfiha Nordestina	30	<button>SIM</button>
11	Raízes do Nordeste - Campinas Cambui	Cuscuz Recheado de Frango	35	<button>SIM</button>
12	Raízes do Nordeste - Campinas Cambui	Tapioca de Carne Seca	42	<button>SIM</button>

Figura 33 - Página de visualização de estoque pelo administrador

Fonte: Elaborado pelo autor (2026).

6.12 Pedidos



Admin
Raízes do Nordeste

Painel Administrativo

Pedidos

Acompanhe os pedidos realizados pelos clientes.

Lista de pedidos

ID	Cliente	Loja	Canal	Status	Total
12	2	Raízes do Nordeste - Campinas Centro	WEB	<button>CRIADO</button>	R\$ 25.40
11	6	Raízes do Nordeste - Campinas Centro	MOBILE	<button>CRIADO</button>	R\$ 21.50
10	10	Raízes do Nordeste - Hortolândia Santana	WEB	<button>ENTREGUE</button>	R\$ 7.50
9	9	Raízes do Nordeste - Hortolândia Olívio	BALCAO	<button>PRONTO</button>	R\$ 22.00
8	8	Raízes do Nordeste - Hortolândia Centro	TOTEM	<button>EM_PREPARO</button>	R\$ 8.50
7	7	Raízes do Nordeste - Sumaré Amizade	WEB	<button>PAGO</button>	R\$ 5.00
6	6	Raízes do Nordeste - Sumaré Rebouças	MOBILE	<button>CRIADO</button>	R\$ 17.00
5	5	Raízes do Nordeste - Sumaré Centro	WEB	<button>ENTREGUE</button>	R\$ 29.90
4	4	Raízes do Nordeste - Campinas Campo Grande	BALCAO	<button>PRONTO</button>	R\$ 9.90
3	3	Raízes do Nordeste - Campinas Taquaral	TOTEM	<button>EM_PREPARO</button>	R\$ 6.50
2	2	Raízes do Nordeste - Campinas Cambui	MOBILE	<button>PAGO</button>	R\$ 21.50
1	1	Raízes do Nordeste - Campinas Centro	WEB	<button>CRIADO</button>	R\$ 18.90

Figura 34 - Página de visualização de pedidos pelo administrador

Fonte: Elaborado pelo autor (2026).

6.13 Pagamentos

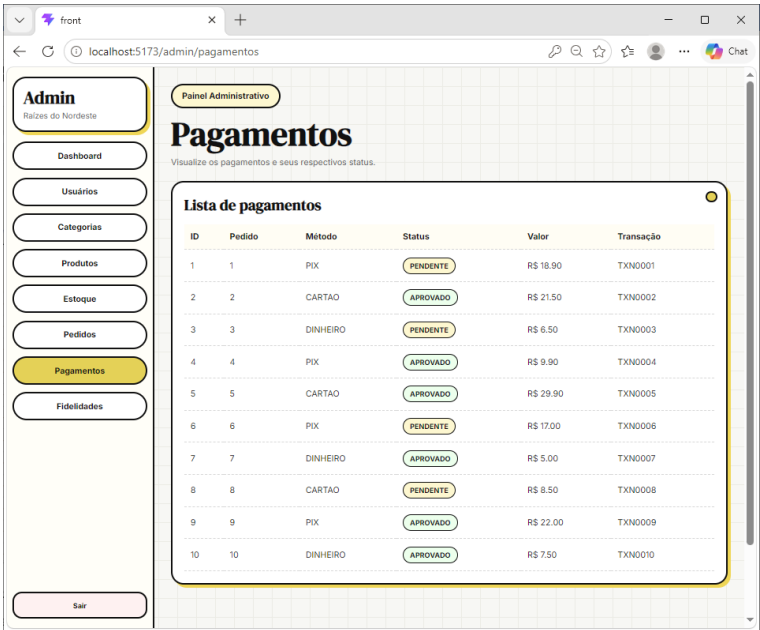


Figura 35 - Página de visualização de pagamentos pelo administrador

Fonte: Elaborado pelo autor (2026).

6.14 Fidelidades

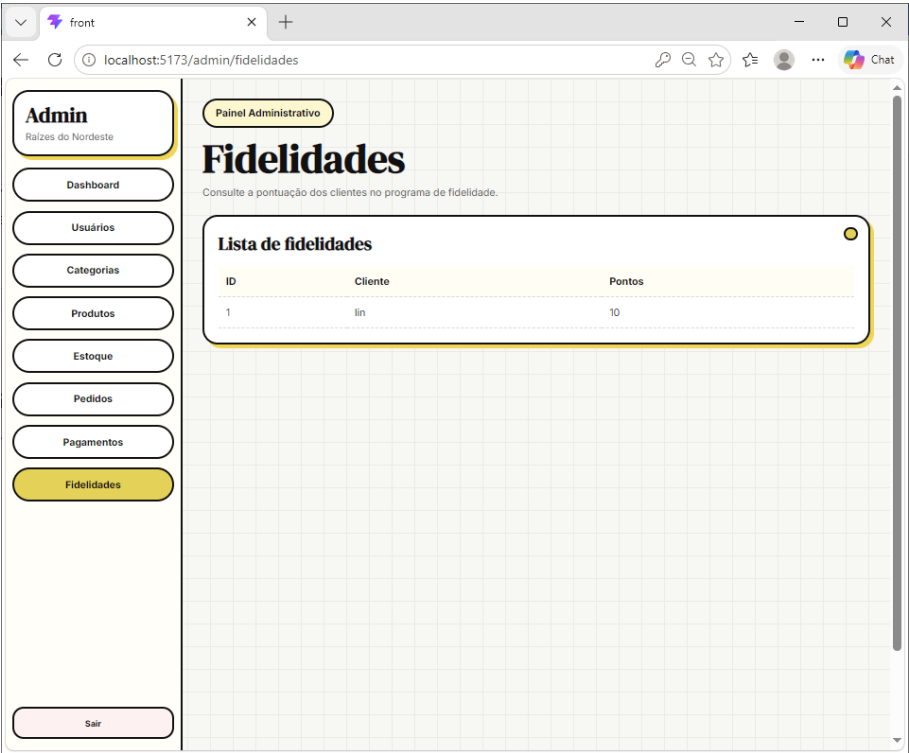


Figura 36 - Página de visualização de fidelidades pelo administrador

Fonte: Elaborado pelo autor (2026).

7 PLANO DE TESTES E EVIDÊNCIAS DO FRONTEND MOBILE

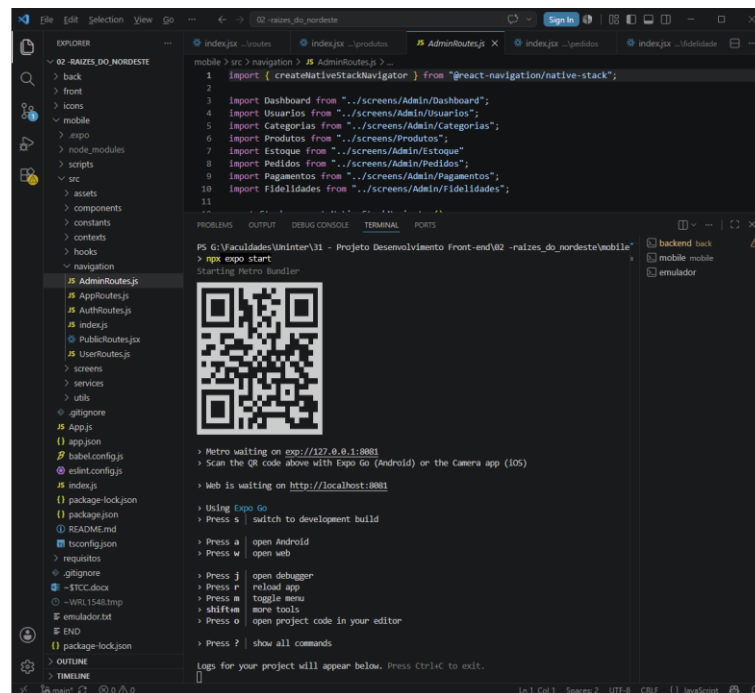


Figura 37 - Iniciando o mobile utilizando Expo

Fonte: Elaborado pelo autor (2026).

7.1 Login de usuário



Figura 38 - Tela de login

Fonte: Elaborado pelo autor (2026).

7.2 - Carrinho



Figura 39 - Tela do carrinho do usuário

Fonte: Elaborado pelo autor (2026).

7.3 Produtos



Figura 40 - Tela de produtos do usuário

Fonte: Elaborado pelo autor (2026).

7.4 Usuário logado

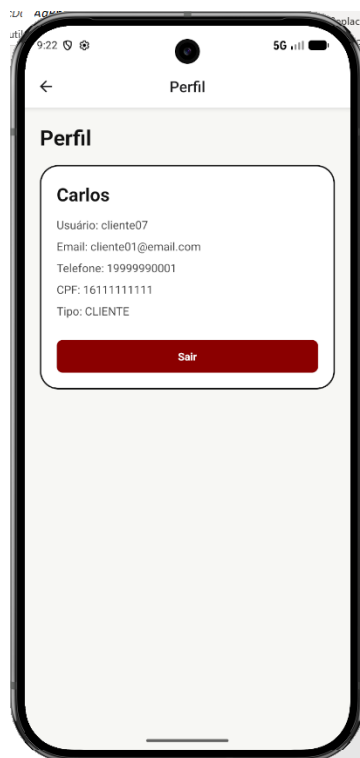


Figura 41 - Tela do perfil do usuário

Fonte: Elaborado pelo autor (2026).

7.5 Login de administrador



Figura 42 - Login como administrador

Fonte: Elaborado pelo autor (2026).

7.6 Painel administrativo (administrador do sistema)



Figura 43 - Tela principal administrativa

Fonte: Elaborado pelo autor (2026).

7.7 Usuários



Figura 44 - Tela de usuários

Fonte: Elaborado pelo autor (2026).

7.8 Categorias

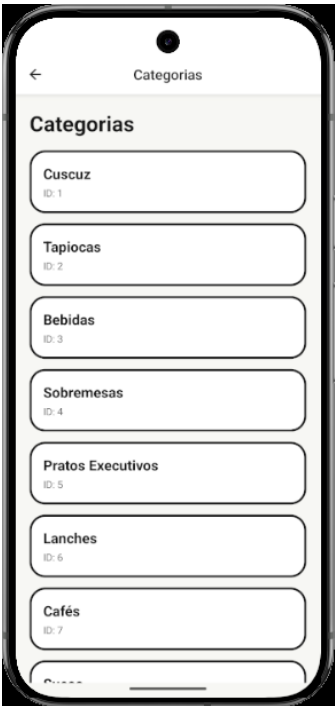


Figura 45 - Tela de categorias

Fonte: Elaborado pelo autor (2026).

7.9 Produtos



Figura 46 - Tela de produtos

Fonte: Elaborado pelo autor (2026).

7.10 Estoque



Figura 47 - Tela de estoque

Fonte: Elaborado pelo autor (2026).

7.11 Pedidos



Figura 48 - Tela de pedidos

Fonte: Elaborado pelo autor (2026).

7.12 Pagamentos



Figura 49 - Tela de pagamentos

Fonte: Elaborado pelo autor (2026).

7.13 Fidelidades



Figura 50 - Tela de fidelidades

Fonte: Elaborado pelo autor (2026).

8 PLANO DE TESTES E EVIDÊNCIAS

8.1 Estratégia de Validação

A validação do sistema Raízes do Nordeste foi planejada com foco na trilha de Back-end, considerando principalmente o funcionamento da API REST, a persistência dos dados, a autenticação dos usuários e a criação de pedidos.

Foram definidos cenários de teste positivos e negativos, com o objetivo de verificar se o sistema responde corretamente tanto em situações esperadas quanto em situações de erro. Os testes podem ser executados por meio de ferramentas como Insomnia ou Postman, utilizando os endpoints disponibilizados pela API.

A estratégia de testes contempla:

- Testes de autenticação;
- Testes de cadastro de usuários;
- Testes de listagem de produtos;
- Testes de criação de pedidos;
- Testes de consulta de pedidos;
- Testes de acesso sem autenticação;
- Testes de envio de dados inválidos.

8.2 Cenários de Teste

ID	Cenário	Entrada	Passos	Saída Esperada	Tipo
CT01	Cadastro de usuário válido	username, senha, e-mail, telefone e CPF	Enviar requisição POST para /api/usuarios/ com dados válidos	Usuário criado com status HTTP 201	Positivo
CT02	Cadastro sem usuário	Dados sem o campo username	Enviar requisição POST para /api/usuarios/ sem username	Retorno de erro informando campo obrigatório	Negativo
CT03	Login válido	username e senha corretos	Enviar POST para /api/token/	Retorno dos tokens access e refresh com status HTTP 200	Positivo
CT04	Login inválido	username ou senha incorretos	Enviar POST para /api/token/ com dados inválidos	Retorno de erro de autenticação com status HTTP 401	Negativo

CT05	Listagem de produtos	Nenhuma entrada obrigatória	Enviar GET para /api/produtos/	Lista de produtos cadastrados	Positivo
CT06	Consulta de perfil autenticado	Token JWT válido	Enviar GET para /api/usuarios/me/ com Authorization Bearer	Retorno dos dados do usuário logado	Positivo
CT07	Consulta de perfil sem token	Nenhum token enviado	Enviar GET para /api/usuarios/me/ sem autenticação	Retorno HTTP 401 informando credenciais não fornecidas	Negativo
CT08	Criação de pedido válido	loja, canal e itens	Enviar POST para /api/pedidos/ com token válido e itens no corpo da requisição	Pedido criado com status HTTP 201	Positivo
CT09	Criação de pedido sem loja	canal e itens, sem loja	Enviar POST para /api/pedidos/ sem o campo loja	Retorno de erro informando que loja é obrigatória	Negativo
CT10	Criação de pedido sem autenticação	loja, canal e itens	Enviar POST para /api/pedidos/ sem token	Retorno HTTP 401 informando credenciais não fornecidas	Negativo
CT11	Consulta de pedidos	Token JWT válido	Enviar GET para /api/pedidos/	Lista de pedidos do usuário autenticado	Positivo
CT12	Produto inexistente no pedido	ID de produto inválido	Enviar POST para /api/pedidos/ com produto inexistente	Retorno de erro ou falha de validação	Negativo

Tabela 7 - Cenários de Teste

8.3 Critérios de Aceitação

O sistema será considerado validado quando:

- Permitir cadastro de usuários com dados válidos;
- Impedir cadastro com campos obrigatórios ausentes;
- Realizar login e retornar token JWT;
- Bloquear acesso a rotas protegidas sem autenticação;
- Listar produtos cadastrados;
- Criar pedidos apenas para usuários autenticados;
- Exigir loja, canal e itens para criação de pedido;
- Calcular corretamente o valor total do pedido;
- Exibir os pedidos vinculados ao usuário logado;
- Retornar mensagens de erro adequadas em situações inválidas.

8.4 Evidências de Execução

As evidências de execução devem ser apresentadas no documento final por meio de prints ou registros obtidos durante os testes.

Sugestões de evidências:

Evidência	Descrição
Evidência 01	Print do POST /api/usuarios/ criando usuário com sucesso
Evidência 02	Print do POST /api/token/ retornando access e refresh
Evidência 03	Print do GET /api/produtos/ retornando a lista de produtos
Evidência 04	Print do GET /api/usuarios/me/ retornando dados do usuário logado
Evidência 05	Print do POST /api/pedidos/ criando pedido com sucesso
Evidência 06	Print do GET /api/pedidos/ exibindo os pedidos
Evidência 07	Print de erro 401 ao acessar rota protegida sem token
Evidência 08	Print de erro ao tentar criar pedido sem informar loja
Evidência 09	Print do aplicativo mobile exibindo produtos
Evidência 10	Print do aplicativo mobile exibindo carrinho e pedidos

5.5 Ferramentas Utilizadas nos Testes

Para a execução dos testes, podem ser utilizadas as seguintes ferramentas:

- Insomnia: para testar os endpoints da API;
- Postman: para criação de coleções de testes;
- SQLite Browser: para verificar os registros gravados no banco;
- Expo Go: para validar o funcionamento do aplicativo mobile;
- Console do Django: para acompanhar requisições e erros;
- Logs do React Native: para verificar respostas da API e falhas de autenticação.

8.6 Exemplo de Teste com Insomnia

Teste: Login de Usuário

Método:

POST

Endpoint: /api/token/

Corpo da requisição:

```
{
  "username": "cliente01",
  "password": "123456"
}
Resultado esperado:
{
  "access": "token_de_acesso",
  "refresh": "token_de_atualizacao"
}
```

Critério de aceitação:

O sistema deve retornar status HTTP 200 e gerar os tokens de autenticação.

Teste: Criação de Pedido

Método:

POST

Endpoint: /api/pedidos/

Cabeçalho:

Authorization: Bearer token_de_acesso

Corpo da requisição:

```
{
  "loja": 1,
  "canal": "MOBILE",
  "itens": [
    {
      "produto": 2,
      "quantidade": 1
    }
  ]
}
```

Resultado esperado:

O sistema deve criar o pedido, calcular o valor total e retornar os dados do pedido criado.

8.7 Considerações Sobre os Testes

Os testes realizados demonstram que a API atende aos principais requisitos funcionais definidos para o projeto. A autenticação por JWT garante que funcionalidades protegidas sejam acessadas apenas por usuários autenticados, enquanto os testes negativos comprovam que o sistema é capaz de bloquear ações inválidas, como criação de pedidos sem token ou sem informações obrigatórias.

Dessa forma, o plano de testes contribui para validar a confiabilidade da solução e comprovar que o sistema foi desenvolvido considerando não apenas o funcionamento esperado, mas também possíveis falhas de uso.

9. CONCLUSÃO

O desenvolvimento do projeto Raízes do Nordeste proporcionou uma experiência prática e próxima da realidade encontrada no mercado de tecnologia. Ao longo do trabalho, foi possível aplicar conceitos relacionados à análise de requisitos, modelagem de sistemas, arquitetura de software, desenvolvimento de APIs e integração entre diferentes camadas da aplicação, demonstrando que a construção de uma solução tecnológica vai muito além da implementação de código.

Um dos principais aprendizados obtidos durante o projeto foi a importância de compreender o negócio antes de definir qualquer solução técnica. Conforme destacado no estudo de caso, o objetivo não era apenas desenvolver funcionalidades, mas entender os desafios enfrentados pela organização e propor uma solução coerente com suas necessidades atuais e futuras. Essa visão permitiu tomar decisões mais fundamentadas em relação à estrutura do banco de dados, à organização dos serviços da API e ao fluxo de atendimento dos clientes.

Entre os desafios encontrados, destacam-se a modelagem adequada das entidades do sistema, a implementação da autenticação de usuários por meio de tokens JWT, a integração entre o aplicativo mobile e a API REST, bem como o tratamento de erros e validações necessárias para garantir a segurança e a integridade dos dados. Além disso, a necessidade de suportar múltiplos canais de atendimento exigiu uma arquitetura flexível e preparada para futuras expansões.

Os testes realizados contribuíram para validar o funcionamento das principais funcionalidades do sistema, incluindo cadastro de usuários, autenticação, listagem de produtos, gerenciamento do carrinho de compras e criação de pedidos. Os resultados obtidos demonstraram que a solução atende aos requisitos propostos e possui uma estrutura consistente para utilização em cenários reais.

Como possibilidades de evolução futura, o sistema pode receber novas funcionalidades, como integração com gateways de pagamento, programa completo de fidelidade, notificações em tempo real sobre o andamento dos pedidos, integração com plataformas de delivery, geração de relatórios gerenciais e dashboards para acompanhamento de indicadores de desempenho. Também seria possível ampliar a infraestrutura para suportar maior volume de acessos simultâneos, utilizando serviços em nuvem e mecanismos de balanceamento de carga.

Por fim, conclui-se que os objetivos propostos para o projeto foram alcançados, resultando em uma solução capaz de atender às necessidades da rede Raízes do Nordeste e, ao mesmo tempo, proporcionar uma importante experiência de desenvolvimento profissional. O trabalho permitiu consolidar conhecimentos técnicos e fortalecer competências fundamentais

para atuação no mercado de tecnologia, como análise crítica, resolução de problemas, documentação e tomada de decisões arquiteturas.

10 REFERÊNCIAS BIBLIOGRÁFICAS

DJANGO REST FRAMEWORK. *Django REST framework*. Disponível em: <https://www.django-rest-framework.org/>. Acesso em: 23 jun. 2026.

META PLATFORMS. *React Documentation*. Disponível em: <https://react.dev/>. Acesso em: 23 jun. 2026.

EXPO. *Expo Documentation*. Disponível em: <https://docs.expo.dev/>. Acesso em: 23 jun. 2026.

SQLITE. *SQLite Documentation*. Disponível em: <https://www.sqlite.org/docs.html>. Acesso em: 23 jun. 2026.

DJANGO SOFTWARE FOUNDATION. Django Documentation: Models. Disponível em: <https://docs.djangoproject.com/en/stable/topics/db/models/>. Acesso em: 24 jun. 2026.

ANEXO A – Manual de Instalação

Para executar o sistema localmente, são necessários os seguintes softwares:

- Python 3.13 ou superior
- Node.js 22 ou superior
- Git
- Expo Go (Android)
- SQLite

Back-end

1. Clonar o repositório.
2. Criar ambiente virtual.
3. Instalar as dependências.
4. Executar as migrações.
5. Iniciar o servidor.

Front-end Web

1. Acessar a pasta front.
2. Instalar as dependências.
3. Executar o projeto utilizando `npm run dev`.

Front-end Mobile

1. Acessar a pasta mobile.
2. Instalar as dependências.
3. Executar o projeto utilizando `npx expo start`.

Após esses procedimentos, todas as aplicações estarão aptas para comunicação com a API REST desenvolvida em Django.

Anexo B – Endpoints completos

Alguns endpoints no corpo do texto.

Exemplo:

Método	Endpoint	Autenticação
POST	/api/token	Não
POST	/api/token/refresh	Sim
GET	/api/produtos	Não
GET	/api/categorias	Não
GET	/api/lojas	Não
GET	/api/pedidos	Sim
POST	/api/pedidos	Sim
GET	/api/usuarios/me	Sim

Anexo C – Estrutura do Banco

Todas as tabelas:

usuarios_usuario

catalogo_categoria

catalogo_produto

catalogo_estoque_loja

pedidos_pedido

pedidos_item_pedido

pagamentos_pagamento

lojas_loja

fidelidade_fidelidade

Anexo D – Tecnologias

Uma tabela.

Tecnologia	Finalidade
Python	Backend
Django	API
DRF	REST
JWT	Segurança
React	Front Web
React Native	Mobile
Expo	Execução Mobile
Firebase	Hospedagem Front
PythonAnywhere	Hospedagem API
SQLite	Banco

Anexo E – Mobile

Link abaixo para fazer download da aplicação mobile:



<https://expo.dev/accounts/lintelecom/projects/mobile/builds/8e673479-67c8-4a8e-b927-99f5c5edbffc>

Anexo F – WEB

Link para acessar a web:

<https://raizesdonordeste4427651.web.app/admin>

Anexo F – Repositório

https://github.com/lindomarbatastao/raizes_do_nordeste.git